

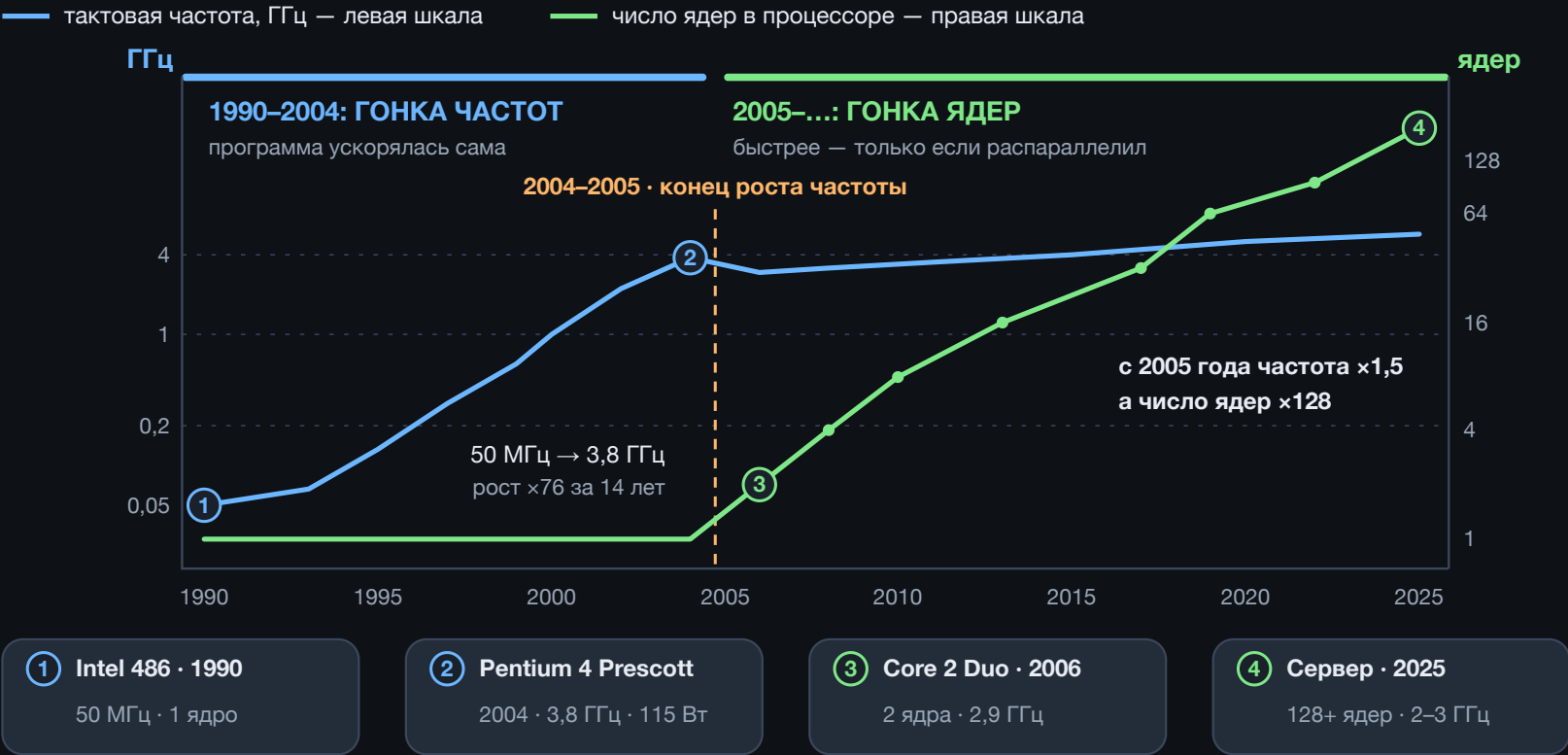
0. Зачем нам конкурентность

Многопоточное программирование — от потока до кеш-линии

- 0. Зачем всё это: конкурентность, параллелизм, законы масштабирования
- 1–2. Процессы, потоки, планировщик ОС и первые гонки
- 3–4. Синхронизация и каналы: как писать правильно
- 5–6. Атомики, lock-free и модель памяти
- 7. Кеш процессора — гвоздь программы
- 8. Практика, инструменты, здравый смысл

Двадцать лет назад код ускорился сам

ЧАСТОТА УПЁРЛАСЬ В СТЕНУ — ДАЛЬШЕ РАСТЁМ ТОЛЬКО ВШИРЬ



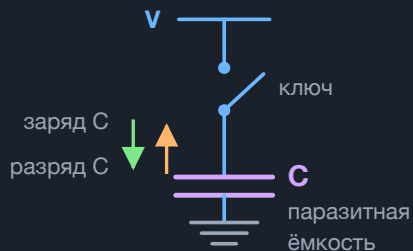
Частота стоит уже двадцать лет. Растёт только число ядер — а пользоваться ими надо руками.

До 2005-го за скорость вашей программы отвечал завод, выпускавший процессоры. С тех пор — вы сами.

Почему гигагерцы кончились

ФИЗИКА СТЕНЫ МОЩНОСТИ: ПОЧЕМУ ГИГАГЕРЦЫ КОНЧИЛИСЬ

1 ТРАНЗИСТОР КАК КЛЮЧ



каждое переключение стоит энергии

$$E = C \cdot V^2 / 2$$

переключений в секунду — f

$$P = C \cdot V^2 \cdot f$$

динамическая мощность

2 МАСШТАБИРОВАНИЕ ДЕННАРДА, до 2005

транзистор уменьшили в k раз —
и вместе с ним поехало всё:

↓ размер	$\div k$
↓ напряжение V	$\div k$
↓ энергия $C \cdot V^2 / 2$	$\div k^3$
= мощность на мм^2	та же

транзисторов на мм^2 — в k^2 больше,
но каждый ест в k^2 меньше

→ **греется ровно так же**
частота росла бесплатно

3 ПОЧЕМУ ЭТО СЛОМАЛОСЬ в 2005

напряжение перестало падать
ниже порога транзистор просто
не открывается — и течёт закрытым

растить можно только f
мощность идёт за частотой линейно,
а за напряжением — квадратично

$f \times 2 \rightarrow P \times 2$ линейно

$V \times 1,2 \rightarrow P \times 1,44$ квадратично

горячее → утечка больше → ещё горячее
положительная обратная связь по теплу

Pentium 4 Prescott: 115 Вт на кристалле $\approx 1,1 \text{ см}^2$ — это около 100 Вт/см^2

у конфорки электроплиты 5–10 Вт/см². Кремний греется в десять раз плотнее.

Дальше греть нечем — пошли вширь, в ядра

Тридцать лет физика бесплатно ускоряла наш код. В 2005-м она прислала счёт.

Половина частоты — восьмая мощности

- $P \sim C \cdot V^2 \cdot f$: частота входит линейно, напряжение — в квадрате.
- Чтобы поднять частоту, приходится поднимать и напряжение: выходит почти куб.
- Снизили f и V вдвое — мощность упала в восемь раз, скорость только вдвое.
- На тот же бюджет ватт влезает четыре медленных ядра вместо одного.
- Отсюда же деление на производительные и энергоэффективные ядра в одном чипе.

Многоядерность — не мода архитекторов, а прямое следствие квадрата в формуле.

Закон Мура жив. Бесплатный обед — нет

- Закон Мура — про **число транзисторов**, а не про скорость одного потока.
- Транзисторы всё ещё удваиваются. Скорость одного потока — давно нет.
- Иссякли три источника даровой скорости:
 - частота — упёрлась в мощность и тепло;
 - внеочередное исполнение — упёрлось в свой потолок;
 - рост кеша — помогает, но больше не удваивает.

«*The free lunch is over. Now welcome to the hardware jungle.*» — Herb Sutter, 2005

Куда ушли транзисторы

- **Больше ядер.** Ноутбучный чип — 10 ядер, серверный сокет — 128–256.
- **Шире SIMD.** Одна инструкция считает 8–16 чисел сразу.
- **Больше кеша.** Мегабайты на кристалле, чтобы реже ходить в память.
- **Спекуляция.** Сотни инструкций «в полёте» ради одного-единственного потока.
- Но ядро не разложит ваш алгоритм по десяти ядрам само.

Производительность больше не выдают бесплатно — её теперь программируют.

Счёт на сегодня: 5,7 ГГц против 256 ядер

- За двадцать лет частота выросла в полтора раза: 3,8 → 5,7 ГГц.
- Число ядер за те же годы: 1 → 256 на сокет.
- AMD EPYC Venice (Zen 6): 256 ядер, 512 потоков, техпроцесс 2 нм.
- Intel Xeon 7 Diamond Rapids: до 256 ядер, анонсирован, ожидается 2026–2027.
- Ноутбук лектора: 10 ядер — 4 производительных и 6 энергоэффективных.

Вопрос не «нужна ли многопоточность», а «сколько ядер вы уже не используете».

Конкурентность и параллелизм — не одно и то же

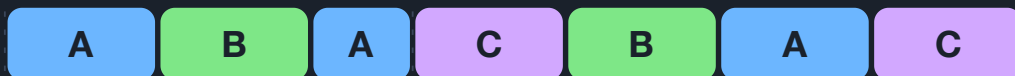
КОНКУРЕНТНОСТЬ ≠ ПАРАЛЛЕЛИЗМ

— две разные вещи, которые постоянно путают

КОНКУРЕНТНОСТЬ — один исполнитель, три задачи по очереди

■ задача А ■ задача В ■ задача С

ядро



в каждый момент времени работает ровно одна задача

аналогия: один бариста на три заказа

ПАРАЛЛЕЛИЗМ — два исполнителя, задачи идут одновременно

ядро 1



ядро 2

две задачи реально выполняются в одну и ту же секунду

аналогия: два бариста за одной стойкой

→ время

ФОРМУЛИРОВКА РОБА ПЙКА

Конкурентность —
это про то, как
устроена программа.

Параллелизм —
про то, как она
исполняется.

конкурентность —
как написан код

параллелизм —
сколько ядер реально

Конкурентным можно быть и на одном ядре. Параллельным — только если ядер больше одного.

«Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once.» — Rob Pike, 2012

Конкурентность проектируют, параллелизм выдаёт железо

- Один и тот же код: на одном ядре — переключение, на десяти — реальное ускорение.
- Параллельно, но не конкурентно: SIMD и GPU — один поток, много данных.
- Конкурентно, но не параллельно: **событийный цикл** — один поток опрашивает тысячи сокетов.
- Дальше всё решает один вопрос: программа считает — или ждёт?

Мы считаем или мы ждём?

- **CPU-bound** — упираемся в арифметику: матрицы, видео, хеши, сжатие.
- Лечится настоящими ядрами и дружбой с кешем (часть 7).
- **IO-bound** — почти всё время ждём чужого ответа:
 - NVMe-диск — десятки-сотни микросекунд на запрос;
 - сеть внутри дата-центра — сотни микросекунд, между континентами — ~150 мс.
- Ждать умеет и одно ядро: здесь выигрывает асинхронность, а не число ядер.

Одна идея — три исполнителя

// C++: поток ОС на клиента. Под стек резервируется ~8 МБ виртуальной памяти
`std::thread(serve, accept(fd)).detach();`

CPP

// Go: горутина на клиента. Стек стартует с 2–8 КБ и растёт по надобности
`go serve(conn)`

GO

Python: исполнителя на клиента нет вообще — один поток и событийный цикл
`asyncio.create_task(serve(conn))`

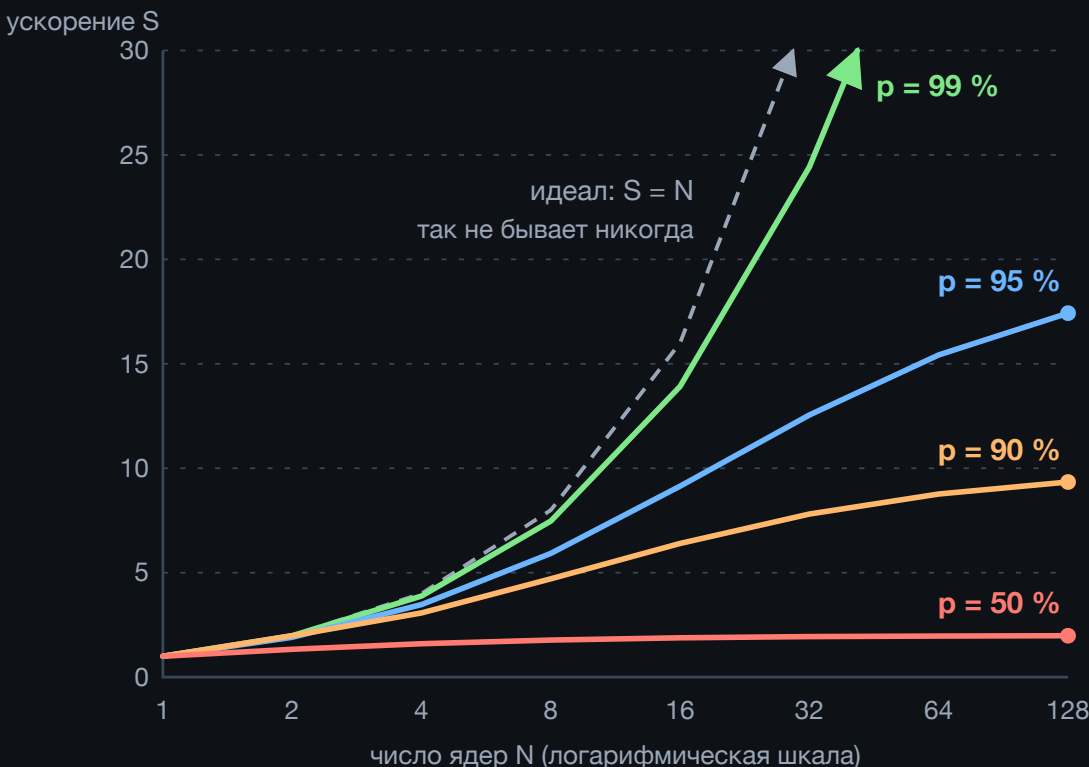
PYTHON

- Проблема C10k (1999): десять тысяч соединений на одной машине.

Закон Амдала: потолок ускорения

ЗАКОН АМДАЛА: ПОТОЛОК УСКОРЕНИЯ

— почему 64 ядра не дают ×64



$$S(N) = 1 / ((1-p) + p/N)$$

p — доля кода, которая параллелизуется

$p = 50\%$	потолок	× 2
$p = 90\%$	потолок	× 10
$p = 95\%$	потолок	× 20
$p = 99\%$	потолок	× 100
---	идеал $S = N$	потолка нет

5 % последовательного кода — это потолок ×20 навсегда

сколько ядер ни добавляй

- Потолок задаёт не число ядер, а ваша доля p — и её надо измерить, а не угадать.

Где прячется последовательная часть

- Инициализация, чтение конфига, прогрев, сборка результатов в один массив.
- Критическая секция под общим мьютексом: пока внутри один — остальные стоят.
- Общие на всех аллокатор, логгер, метрики, счётчик запросов.
- Синхронизация не бесплатна, и с ростом числа потоков дорожает.
- Амдал ещё и оптимист: обмен данными между потоками он не считает вовсе.
- Правило: сначала уменьшай последовательную часть, потом добавляй потоки.

Закон Густафсона: не всё так мрачно

- Амдал фиксирует задачу: «во сколько раз быстрее посчитаем то же самое?».
- Густафсон фиксирует время: «насколько больше успеем за те же секунды?».
- Масштабированное ускорение: $S(N) = (1 - p) + p * N$.
- Получив ядра, мы обычно берём задачу крупнее, а не ждём ответ быстрее.
- Ночной отчёт по вчерашним данным — Амдал. Растущий поток запросов — Густафсон.

Чем мы платим за конкурентность

- **Недетерминизм**: падает раз в тысячу запусков и только на проде.
- **Гонки данных (data race)**: в C++ это неопределённое поведение (UB).
- **Дедлоки (deadlock)**, лайвлоки, инверсия приоритетов, утечки потоков.
- **Отладка**: отладчик и `printf` меняют тайминги, и баг прячется.
- **Цена синхронизации**: бывает, что два потока работают медленнее одного.

Сначала спроси: а нужно ли?

- Смена алгоритма обычно даёт больше, чем десять потоков.
- Кеш-дружелюбный обход данных ускоряет в разы без единого потока.
- Пачечная обработка (batching) убирает накладные расходы дешевле параллелизма.
- Если профиль показывает ожидание — потоки не помогут, убирайте ожидание.
- Многопоточность — обмен простоты на скорость, и курс бывает грабительский.

Три способа делать несколько дел сразу

- **Процессы** — изоляция, крах одного не роняет остальных; дорогое переключение.
 - так устроены веб-сервер nginx и СУБД PostgreSQL
- **Потоки** — общая память, дешёвое создание и общение; платим гонками.
 - так устроены игровые движки, браузеры, сервисы на JVM
- **Событийный цикл** — один поток и `epoll / kqueue`, тысячи соединений.
 - минус: один блокирующий вызов вешает всё, счётная работа не ускоряется
- **Гибрид**: горутины Go — модель M:N, много лёгких задач на немногих потоках ОС.

Карта лекции: мы здесь

- **0. Зачем всё это** — закончили: стена частот, Амдал, цена конкурентности.
- **1. Процессы, потоки, планировщик** — идём туда прямо сейчас.
- 2–4. Гонки, мьютексы, каналы: как писать правильно.
- 5–6. Атомики, lock-free, модель памяти: почему код выполняется не как написан.
- 7. Кеши: локальность, когерентность, false sharing — гвоздь программы.
- 8. Практика: правила, инструменты, тестирование, что почитать.

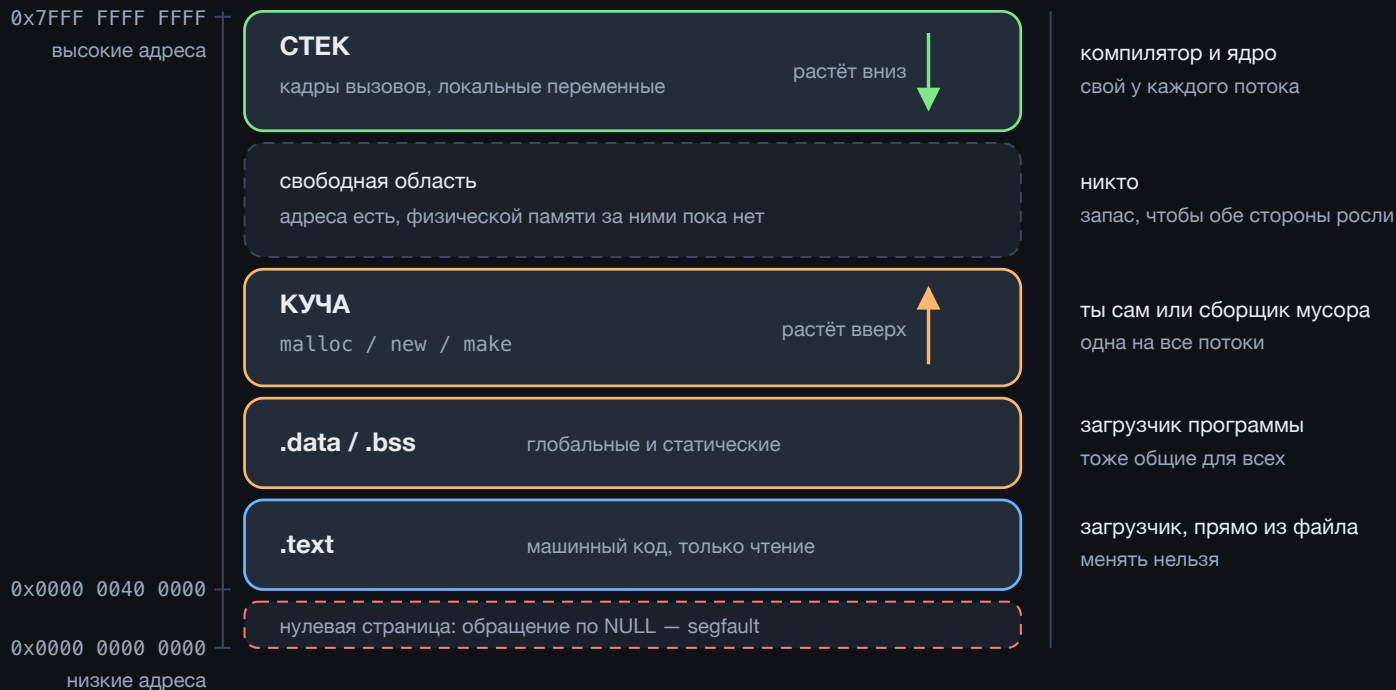
1. Процессы, потоки и планировщик

- Кто на самом деле исполняет наш код
- Процесс, поток, квант, переключение контекста
- Почему «поток на клиента» упирается в стену
- Горутины и модель M:N: как Go эту стену обходит
- Планировщик Go изнутри: GMP и work stealing

Процесс: своя вселенная

АДРЕСНОЕ ПРОСТРАНСТВО ПРОЦЕССА

как программа видит память: одна сплошная линейка байт



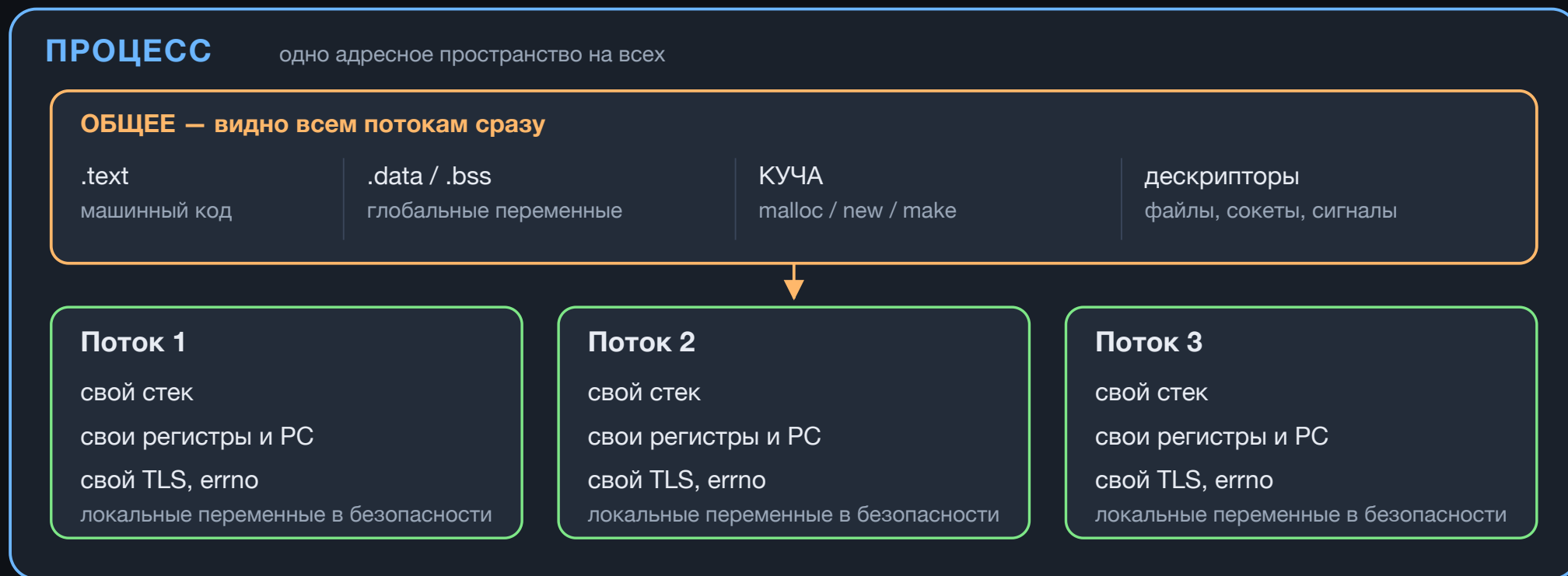
Стек у каждого потока свой, а куча одна на всех
поэтому локальные переменные безопасны, а всё общее нужно синхронизировать

Адресное пространство процесса: стек, куча, данные, код

- Адрес `0x4000` в двух процессах — **разная физическая память**

- Свои дескрипторы файлов, свой PID, свой каталог, свои сигналы

Поток: единица планирования



Общая куча — отсюда растут все гонки данных

два потока пишут в одну переменную, и никто их не синхронизирует

Что потоки делят внутри процесса, а что у каждого своё

- Планировщик кладёт на ядро **поток**, а не процесс

- Процесс сам по себе не исполняется: у него минимум один поток, главный

Общая куча — корень всех проблем

- Потоки одного процесса видят **одну кучу и одни глобалы**
- Передать данные между потоками = передать указатель. Бесплатно
- Значит, два потока могут писать в одну ячейку одновременно
- И никто их не остановит: ни компилятор, ни процессор, ни ОС

Потоки быстры, потому что не изолированы. Потоки опасны, потому что не изолированы. Это одно свойство.

Процесс или поток: за что платим

	Процессы	Потоки
Упал один	соседи живы	умирает весь процесс
Передать данные	мкс плюс сериализация	указатель, наносекунды
Переключение	меняется таблица страниц	меняются только регистры
Утечка памяти	локальна	общая на всех

Как решила индустрия

- **Chrome**: процесс на вкладку и на сайт (site isolation)
- **nginx**: master плюс рабочие процессы, у каждого свой цикл событий
- **PostgreSQL**: процесс на соединение, поэтому в проде ставят пулер
- **JVM, Go, .NET**: один процесс, десятки потоков внутри
- Выбор модели — инженерный компромисс, а не «правильный ответ»

Как они рождаются: один вызов clone

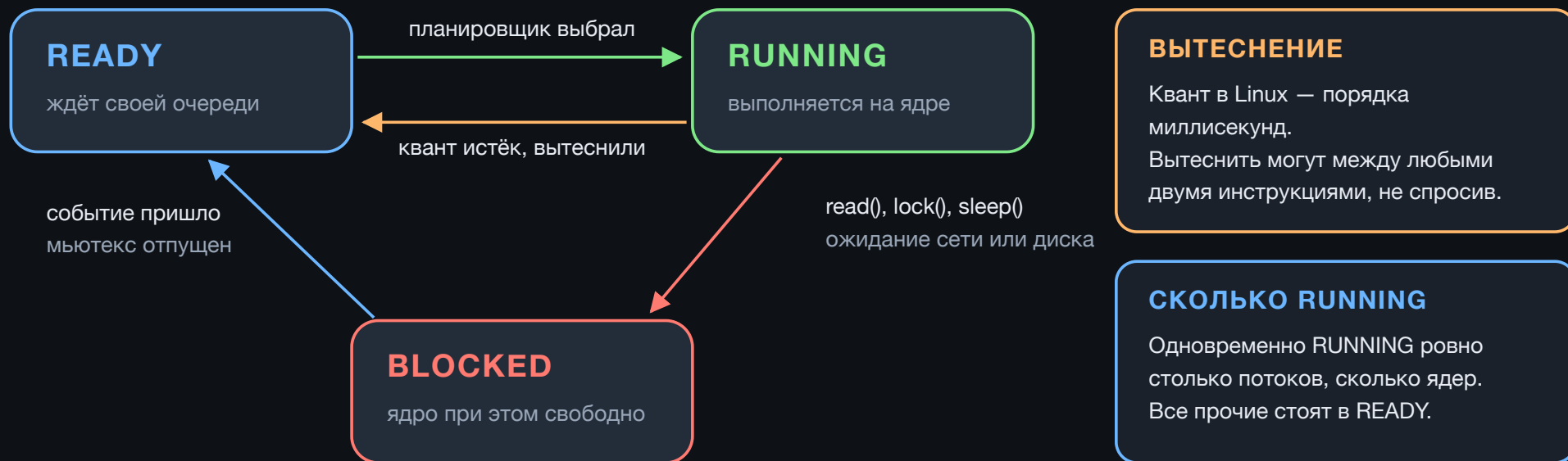
```
fork():          clone(SIGCHLD)                // всё своё
pthread_create(): clone(CLONE_VM | CLONE_FILES |
                       CLONE_SIGHAND | CLONE_THREAD | ...)
```

- `CLONE_VM` («общая память») и есть переключатель «процесс или поток»
- Для ядра Linux это один объект `task_struct` с разным набором общего
- `fork` копирует память лениво: copy-on-write, страницы разъезжаются при записи
- Linux, macOS, Windows — везде модель **1:1**: поток языка равен потоку ядра

Планировщик ОС: разрешения не спрашивает

ТРИ СОСТОЯНИЯ ПОТОКА

переходами управляет планировщик, а не ваш код



Между любыми двумя вашими строчками поток может встать и простоять миллисекунды

поэтому «почти одновременно» и «успеет проскочить» — не аргументы

Три состояния потока и переходы, которыми управляет планировщик

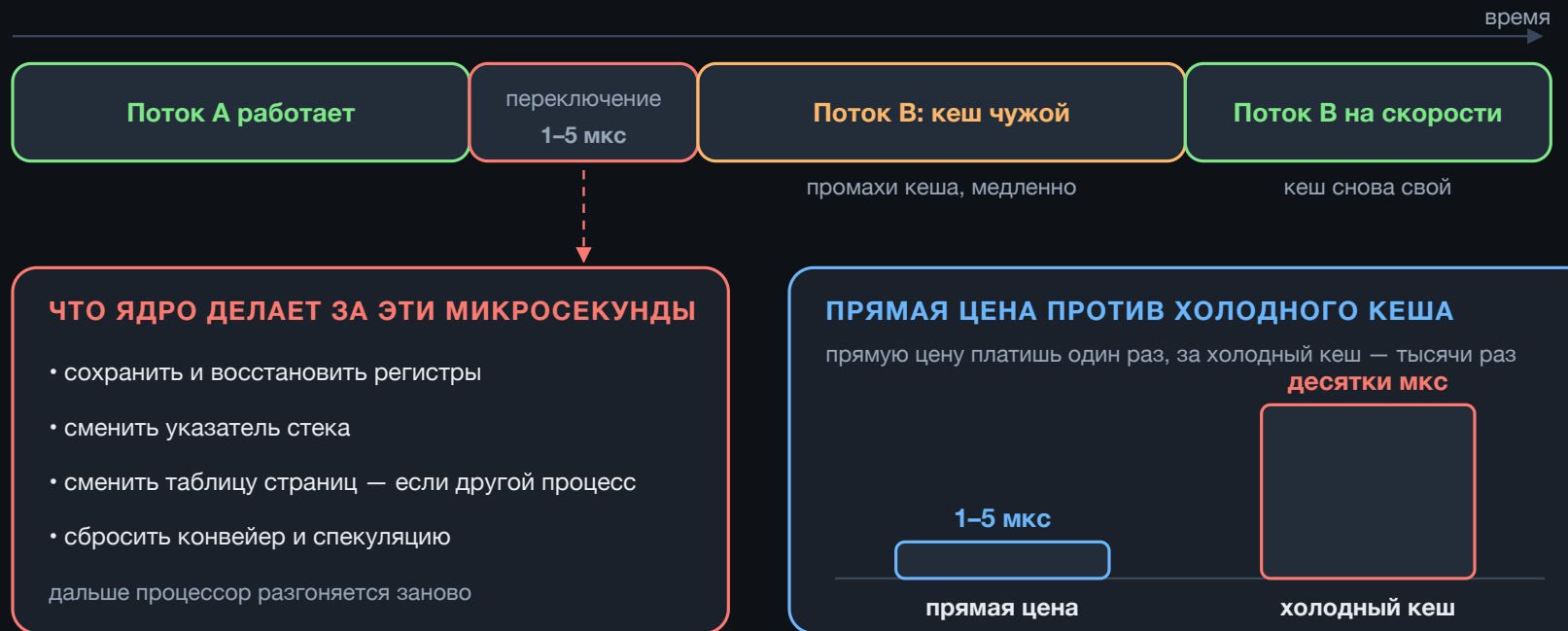
Квант: сколько дают и кто отбирает

- Квант не фиксирован: Linux CFS, с ядра 6.6 — EEVDF, единицы миллисекунд
- У каждого ядра своя очередь готовых плюс балансировка между ядрами
- Ядра бывают разные: на M4 их 10 — 4 производительных и 6 экономичных
- Куда уедет поток, решает подсказка QoS, а не ваше желание
- Из-за этого одинаковый код меряется по-разному от запуска к запуску

Переключение контекста: счёт приходит дважды

ЦЕНА ПЕРЕКЛЮЧЕНИЯ КОНТЕКСТА

одно ядро CPU, время слева направо



Прямая цена — микросекунды. Косвенная — гораздо больше.

кеш и TLB после переключения заполнены чужими данными: первые тысячи обращений нового потока идут в память, а это сотни тактов каждое

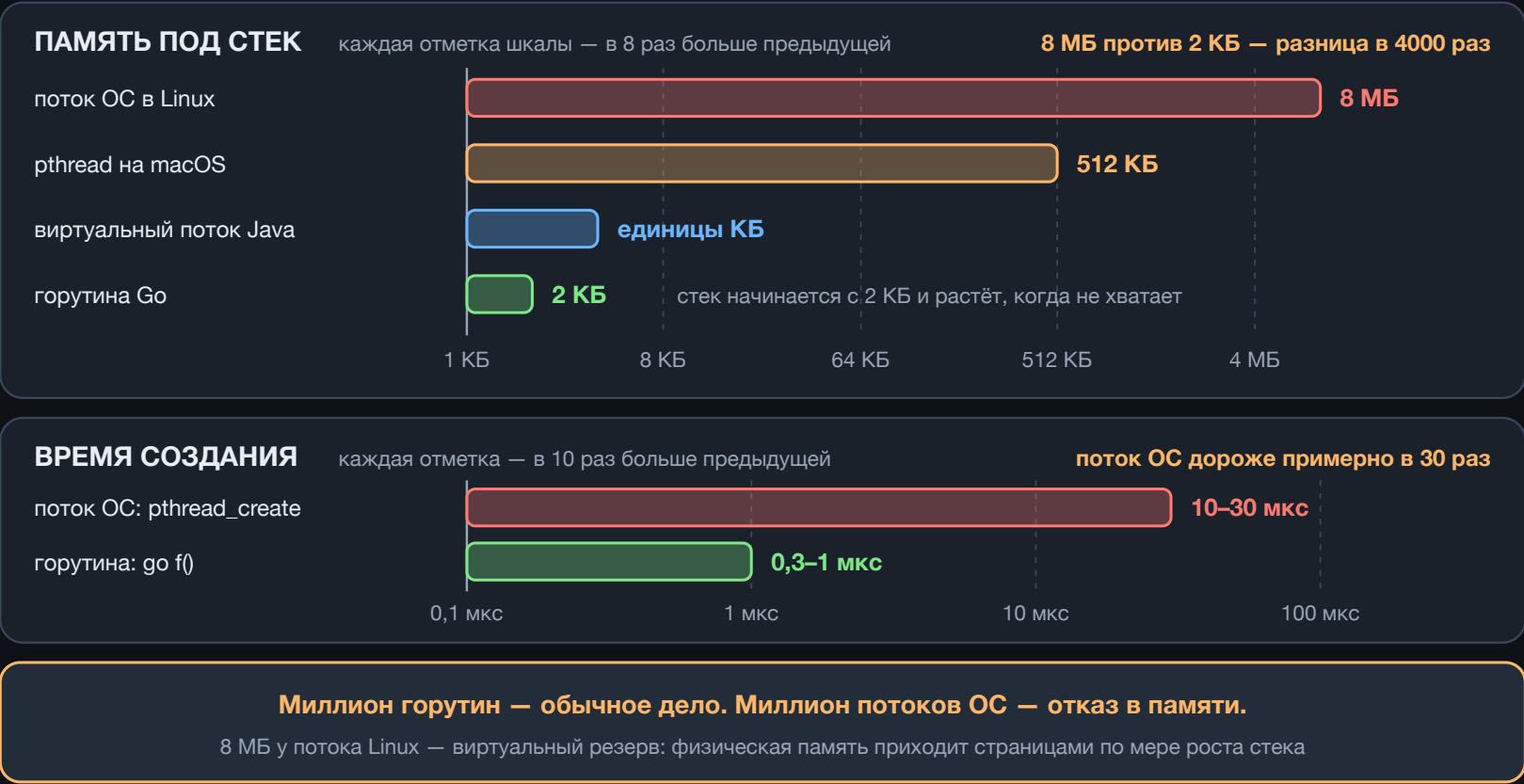
Прямая цена переключения и цена холодного кеша

- Мерить: `perf stat -e context-switches,cache-misses, vmstat 1`, колонка `cs`

Сколько стоит один поток ОС

ЦЕНА ЕДИНИЦЫ ПАРАЛЛЕЛИЗМА

шкала логарифмическая — иначе столбик горутины не было бы видно



Память под стек и время создания: поток ОС против горутины

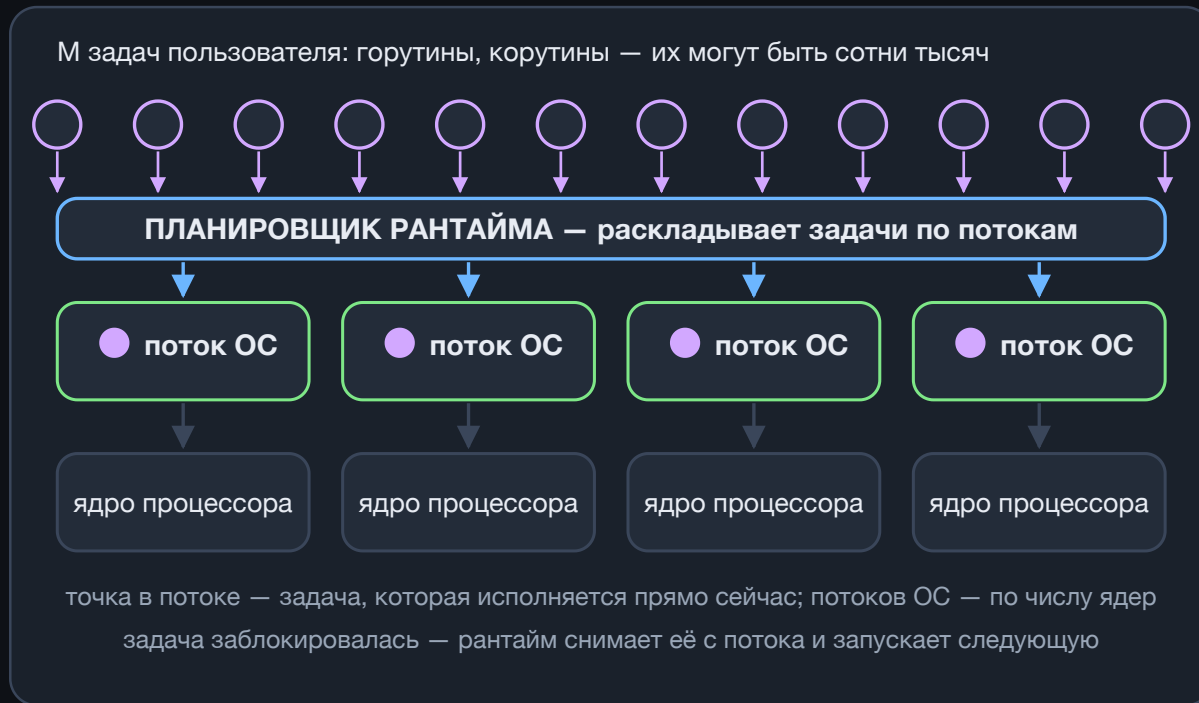
Стена C10K

- 10 000 потоков в Linux — это около 80 ГБ виртуального пространства
- И 10 000 задач, которые планировщик обязан честно перебирать
- **C10K** (Дэн Кегель, 1999): как удержать 10 000 соединений на одной машине
- Ответ индустрии — мультиплексирование: `epoll` в Linux, `kqueue` в BSD и macOS
- Один поток тянет тысячи сокетов, но код превращается в лапшу из колбэков

Пользовательские потоки: модель M:N

МОДЕЛЬ M:N

M задач пользователя живут на N потоках ОС, и N — маленькое



1:1 против M:N

1:1 — поток ОС на задачу

std::thread, pthread, потоки Java
10 000 задач = 10 000 потоков
переключение идёт в ядро ОС:
~1–3 мкс на каждое

M:N — задачи поверх потоков

горутины Go, виртуальные потоки Java
10 000 задач = 4–8 потоков ОС
переключение внутри рантайма:
~0,2 мкс, ядро не трогаем

Переключение между задачами внутри одного потока ОС не идёт в ядро

сотни наносекунд вместо микросекунд — поэтому задач можно держать десятки тысяч

M задач пользователя живут на N потоках ОС, и N невелико

- Цена — рантайм, который обязан знать про каждый блокирующий вызов

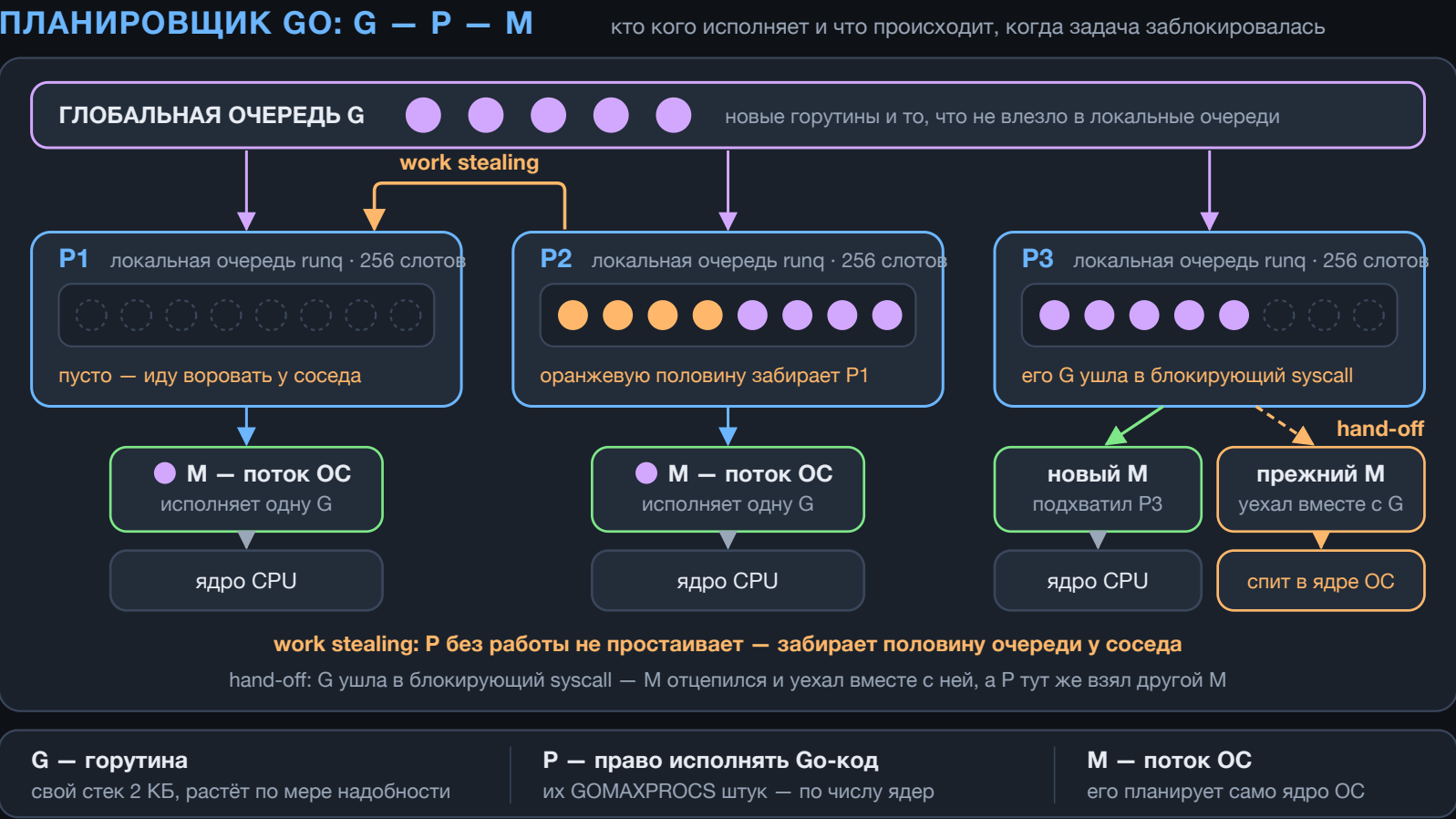
Горутина изнутри: стек 2 КБ, который растёт

```
go worker(i) // ~2 КБ стека, ~0,5 мкс на запуск
fmt.Println(runtime.NumGoroutine()) // сколько их живёт прямо сейчас
```

GO

- `go f()` — не системный вызов, а постановка структуры `g` в очередь
- Стек стартует с **2 КБ** вместо мегабайтов
- Не хватило — рантайм берёт вдвое больший кусок и **копирует** туда старый
- Стек переезжает, поэтому адрес локальной переменной может измениться

Планировщик Go: G, P и M



Планировщик Go: глобальная очередь, work stealing и hand-off

Почему в Go пишут блокирующий код

- Сетевой вызов не блокирует поток ОС: `netpoller` поверх `epoll` и `kqueue`
- Горутина паркуется, поток тут же берёт следующую — колбэков нет
- `sysmon` — фоновый поток без `P`: будит `netpoller`, отбирает `P` после 10 мс
- С Go 1.14 вытеснение асинхронное: потоку шлют сигнал `SIGURG`
- До 1.14 пустой `for {}` намертво занимал `P` и вешал сборщик мусора

Stackful против stackless: одна идея, три языка

```
resp, err := http.Get(url) // Go: обычный вызов, рантайм сам паркует горутину
```

GO

```
resp = await client.get(url) # Python: await, и функция обязана быть async
```

PYTHON

```
let resp = client.get(url).await?; // Rust: автомат состояний генерирует компилятор
```

RUST

- **Stackful** (горутины, Java 21): приостановиться можно с любой глубины
- **Stackless** (`await`, `co_await`): состояние — автомат в куче, рантайм не нужен
- Плата за stackless — «цветные функции»: обычная функция ждать не умеет

Кто во что: сводка моделей

Механизм	Модель	Стек	Переключение
<code>std::thread</code> C++	1:1	0,5–8 МБ	в ядре, 1–5 мкс
горутина Go	M:N	2 КБ, растёт	в рантайме, ~200 нс
virtual thread Java 21	M:N	растёт, в куче	в рантайме
корутина C++20	stackless	кадр в куче	вызов и возврат, нс
async/await Rust, Python	stackless	автомат в куче	вызов <code>poll</code>

Демо: 100 000 против 100 000

```
var wg sync.WaitGroup
for i := 0; i < 100_000; i++ {
    wg.Go(func() { time.Sleep(time.Second) }) // Go 1.25
}
wg.Wait() // отрабатывает; память – сотни МБ, не гигабайты
```

GO

```
std::vector<std::thread> ts;
for (int i = 0; i < 100000; ++i)
    ts.emplace_back([]{ std::this_thread::sleep_for(1s); });
// падает задолго до 100000:
// std::system_error: resource_unavailable_try_again
```

CPP

Мини-итог части

1. Процесс — изоляция и своё адресное пространство, поток — **общая куча**
2. Вытеснить вас могут между любыми двумя инструкциями
3. Переключение стоит микросекунды плюс холодный кеш, и второе дороже
4. Поток ОС дорог по памяти, поэтому «поток на клиента» умирает на C10K
5. M:N уносит планирование в user space: 2 КБ и сотни наносекунд
6. Дёшево создавать задачи не значит безопасно ими пользоваться

2. Первые потоки и первые гонки

- Запускаем поток в C++, Go и Python — и ждём его
- Живая демка: два потока, два миллиона инкрементов, неверный ответ
- Почему `x++` — это три операции, а не одна
- Гонка данных (data race) против состояния гонки (race condition)
- Чем ловить: `-race`, ThreadSanitizer — и сколько это стоит
- Первое лекарство: критическая секция и мьютекс (mutex)

Поток запускается одной строкой

```
std::thread t([]{ std::puts("привет из потока"); });  
t.join(); // ждём, пока он закончит
```

CPP

```
go fmt.Println("привет из горутины") // ждать надо отдельно
```

GO

```
t = threading.Thread(target=work)  
t.start(); t.join()
```

PYTHON

- синтаксис почти одинаковый — вся сложность начинается дальше
- `main` — тоже поток: в каждой из этих программ их два
- порядок строк в выводе не определён, решает планировщик

Дождаться — ваша обязанность

```
std::thread a(work), b(work);  
a.join(); b.join();           // без join — std::terminate
```

CPP

```
var wg sync.WaitGroup  
for i := range 5 {  
    wg.Go(func() { work(i) }) // Go 1.25: Add и Done внутри  
}  
wg.Wait()
```

GO

- выход из `main` в Go равен `os.Exit`: горутины не докручиваются
- их `defer` не выполняются, работа теряется молча
- в C++ забытый `join` — громкое падение, в Go — тихая потеря

C++: забыл join — получил std::terminate

```
{  
    std::jthread t([](std::stop_token st) {  
        while (!st.stop_requested()) step();  
    });  
} // деструктор сам: request_stop(), затем join()
```

CPP

- деструктор `std::thread` зовёт `std::terminate()`, если поток joinable
- язык не угадывает за вас: ждать или отпускать — решение автора
- `detach()` отпускает поток, и ссылки на локальные переменные висят
- исключение или ранний `return` до `join()` — тоже `terminate`
- `std::jthread` (C++20): `join` в деструкторе плюс `stop_token`

Ловушка переменной цикла

```
// до Go 1.22: i одна на весь цикл  
for i := 0; i < 5; i++ {  
    go func() { fmt.Println(i) }()    // часто "5 5 5 5 5"  
}  
// Go 1.22+: i своя на каждой итерации – 0..4 вразнобой
```

GO

- семантика зависит от строки `go 1.22` в `go.mod`, а не от компилятора
- в C++ та же мина: лямбда с `[&]` переживает свою переменную
- берите захват копией или передавайте значение аргументом

Демка: счётчик, который не досчитывает

CPP

```
long long counter = 0;

void worker() {
    for (int i = 0; i < 1'000'000; ++i) ++counter;
}

int main() {
    std::thread a(worker), b(worker);
    a.join(); b.join();
    std::printf("%lld\n", counter);    // ждём 2000000
}
```

```
$ clang++ -std=c++20 -O2 02_data_race.cpp && ./a.out
1237655
$ ./a.out
1046982
```

Х++ — это три операции, а не одна

counter++ — это ТРИ операции процессор не умеет прибавлять прямо в памяти



arm64 Apple M4

```
1 ldr w8, [x9]
2 add w8, w8, #1
3 str w8, [x9]
```

x86-64 Intel / AMD

```
1 mov eax, [counter]
2 add eax, 1
3 mov [counter], eax
```

Между любыми двумя из этих трёх шагов поток могут вытеснить

Инкремент разбирается на load, add в регистре и store обратно в память

- атомарность — свойство инструкции процессора, а не строки исходника

- у x86 форма `add [counter], 1` есть — но внутри те же три шага, без `lock` не атомарна

Как именно теряется инкремент

Как теряется инкремент

оба потока делают `counter++`, а счётчик вырос всего на единицу

ШАГ	ПОТОК А	ПОТОК В	counter в памяти
1	читает counter → 41		41
2		читает counter → 41	41
3	в регистре: $41 + 1 = 42$		41
4		в регистре: $41 + 1 = 42$	41
5	пишет 42 в память		42 записал А
6		пишет 42 в память	42 В пишет поверх

Два инкремента, а прибавилась единица — один потерян

counter был 41, каждый поток прибавил по единице — должно быть 43, а в памяти 42

Оба потока прочитали 41, оба записали 42 — один инкремент исчез

- это потерянное обновление (lost update): работа выполнена и выброшена

- чем горячее переменная, тем чаще шаги совпадают

Data race: формальное определение

Гонка данных — это три условия одновременно:

1. два потока обращаются к **одной и той же** ячейке памяти
 2. хотя бы один из них **пишет**
 3. между обращениями нет отношения happens-before
- убрали любое одно условие — гонки данных нет
 - это не «плохой стиль», а формально определённое свойство программы

Гонка данных ≠ состояние гонки

Гонка данных ≠ состояние гонки

это два разных зверя, и ловятся они по-разному



Санитайзер ловит только гонки данных, логические гонки ловятся головой

- **санитайзер** — детектор гонок, который бежит вместе с программой

- не всякая ошибка порядка — гонка данных, и наоборот

В C++ гонка данных — это UB

```
bool flag = false;           // обычный bool, без atomic

void waiter() {
    while (!flag) { }        // -O2: flag читается один раз в регистр
    work();                  // ...и цикл превращается в while (true)
}
```

CPP

- стандарт разрешает считать, что никто другой в `flag` не пишет
- значит чтение можно вынести из цикла, а проверку выкинуть
- «у меня работало на `-O0`» — это не отсутствие бага, а удача
- лечение: `std::atomic<bool>` (часть 5), мьютекс или канал

А что ломается в других языках

язык	что будет при гонке данных
C++	неопределённое поведение, оптимизатор выкинет ваш код
Go	рванные значения, <code>fatal error: concurrent map writes</code>
Java	UB нет, но поток может годами не видеть чужую запись
Python	GIL спасает объекты, но <code>x += 1</code> всё равно теряется
Rust	не компилируется: <code>&mut</code> нельзя разделить между потоками

Чем ловить гонки: санитайзеры

- Go: `go test -race ./...`, `go run -race` — внутри ThreadSanitizer
- C++: `clang++ -fsanitize=thread -g -O1`; с ASan вместе не собирается
- цена: время **x2–x20**, память **x5–10** — в прод не тащим
- динамический анализ: ловится только реально выполнившийся код
- гоняйте `-race -count=10` и обязательно под нагрузкой

Читаем отчёт -race построчно

```
WARNING: DATA RACE
Write at 0x00c0000160a0 by goroutine 8:
    main.worker()
        /Users/ivanovp/kallak/concurrency/materials/go/02_race/main.go:14
Previous read at 0x00c0000160a0 by goroutine 7:
    main.worker()
        /Users/ivanovp/kallak/concurrency/materials/go/02_race/main.go:14
Goroutine 8 (running) created at:
    main.main()
        main.go:23
Found 1 data race(s)
exit status 66
```

Критическая секция: три требования

- **критическая секция (critical section)** — код, работающий с общими данными
- **взаимное исключение (mutual exclusion)** — внутри не больше одного потока

Любое корректное решение обязано давать все три свойства:

1. mutual exclusion — двое внутрь не заходят
2. progress — свободная секция обязана кого-то впустить
3. bounded waiting — ожидание конечно, поток не голодает

Мьютекс: C++, Go, Rust

```
std::mutex m;  
{  
    std::lock_guard lk(m);    // lock() здесь  
    ++counter;  
}                               // unlock() в деструкторе
```

CPP

```
mu.Lock()  
counter++  
mu.Unlock()           // обычно пишут defer mu.Unlock()
```

GO

```
let mut n = counter.lock().unwrap(); // Mutex<T> владеет данными  
*n += 1;                             // unlock на выходе из области
```

RUST

Мьютекс защищает данные, а не код

```
type Counter struct {  
    mu sync.Mutex // защищает всё, что объявлено ниже  
    n int64  
}  
  
func (c *Counter) Inc() { c.mu.Lock(); defer c.mu.Unlock(); c.n++ }
```

GO

- правило «к этим полям только под этим мьютексом» живёт в головах
- поэтому пишите его комментарием прямо у поля
- в C++ так же: приватный `std::mutex`, публичные методы — единственная дверь
- в Rust правило встроено в тип: `Mutex<T>` владеет данными

Гранулярность: один большой или десять маленьких

- **coarse-grained**: один замок на структуру — просто и надёжно
- **fine-grained**: замок на бакет или шард — больше параллелизма
- мелкие замки лежат рядом и попадают в одну кеш-линию
- такая линия начинает летать между ядрами, и десять замков тормозят
- порядок работы: сначала один мьютекс, потом замер, потом шардинг
- держите секцию короткой: под замком никаких сетевых вызовов и логов

Мини-итог части

- запустить поток легко, дожидаться — ваша обязанность
- `x++` — это три операции, поэтому инкременты теряются
- data race — про память, race condition — про логику
- в C++ это UB, в Go — рваные данные и падение рантайма
- ловим санитайзерами, всегда в CI, ценой $\times 2$ – $\times 20$ по времени
- лечим критической секцией: мьютекс защищает данные, а не код

3. Синхронизация вглубь

- Мьютекс мы уже поставили — и гонка исчезла.
- Взрослый вопрос: **а что там внутри?**
- Разберём: спинлок, futex, `sync.Mutex`, `RWMutex`.
- Потом `condvar`, семафоры, дедлоки и их родня.
- Цель — перестать выбирать примитив наугад.

Что внутри мьютекса: наивный спинлок

```
struct NaiveMutex {  
    std::atomic<bool> locked{false};  
    void lock() { // exchange возвращает СТАРОЕ значение  
        while (locked.exchange(true, std::memory_order_acquire))  
            ; // крутимся, пока занято  
    }  
    void unlock() { locked.store(false, std::memory_order_release); }  
};
```

CPP

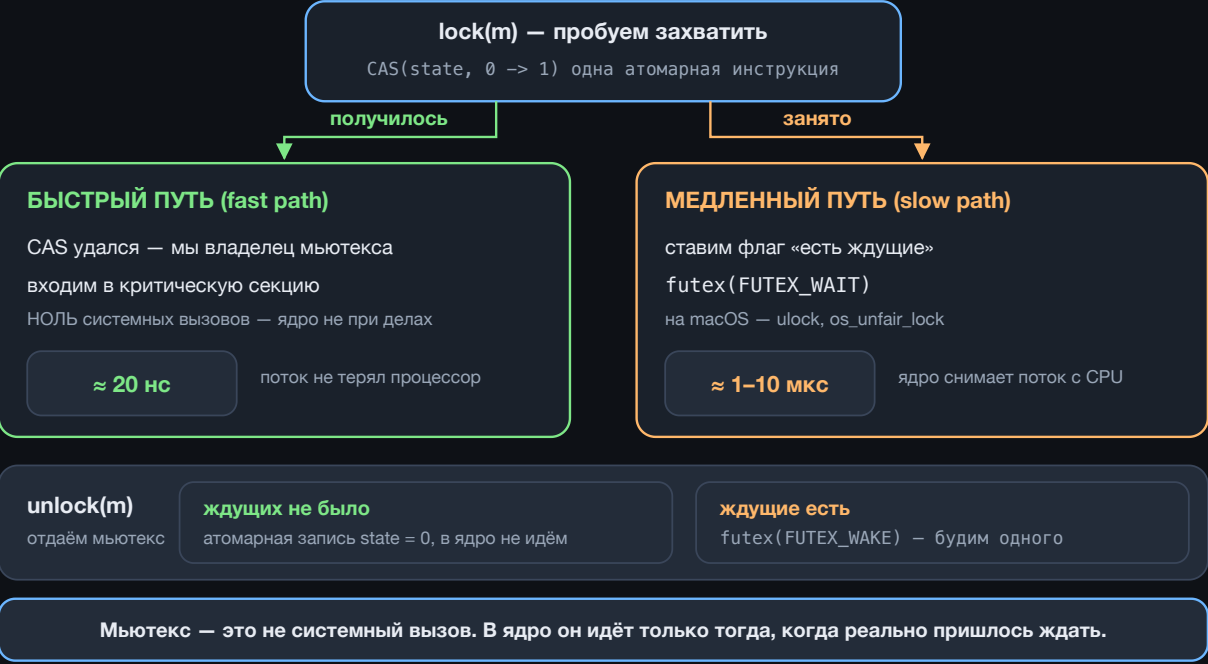
- Весь мьютекс — это одно слово в памяти и атомарный обмен.
- Ноль системных вызовов: ядро ОС вообще ничего не знает.
- Работает корректно. И это худший из возможных мьютексов.

Почему спинлок — плохой мьютекс

- Ждущий поток жжёт такты, не делая ничего полезного.
- Хуже: он мешает владельцу — ворует у него кеш-линию замка.
- Владельца вытеснил планировщик — крутимся весь его квант впустую.
- Один вытесненный владелец на одном ядре = миллисекунды в пустоту.
- Обратный вариант, «сразу спать через ядро», стоит микросекунды.
- Значит, нужен гибрид: быстрый путь без ядра, медленный — с ядром.

Мьютекс: быстрый путь и медленный путь

МЬЮТЕКС: БЫСТРЫЙ ПУТЬ И МЕДЛЕННЫЙ ПУТЬ



В ядро мьютекс идёт только тогда, когда реально пришлось ждать

- **CAS** (compare-and-swap) — атомарное «лежит ожидаемое? тогда положи новое».

- **futex** = fast userspace mutex, Linux, 2002: «усыпи меня на этом адресе».
- Та же идея под другими именами: `os_unfair_lock` в macOS, `SRWLOCK` в Windows.

Спин или парковка: адаптивный мьютекс

- Адаптивная схема: покрутись сотню-другую итераций, потом паркуйся.
- Логика простая: активно ждать выгодно ровно до цены ухода в ядро.
- Правило: спин окупается, если критическая секция короче ~1 мкс.
- Спин обязан быть с паузой: `PAUSE` на x86, `YIELD` / `WFE` на ARM.
- В glibc это `PTHREAD_MUTEX_ADAPTIVE_NP`, в Go — 4 итерации спина.

СКОЛЬКО СТОИТ МЬЮТЕКС НА САМОМ ДЕЛЕ

Операция (без конкуренции)	нс	тактов @4.4 ГГц
обычный <code>++</code> по памяти в L1	0.33	1.4
атомарный <code>fetch_add</code>	1.79	7.9
CAS-цикл	1.82	8.0
<code>pthread_mutex</code> lock + unlock	4.25	18.7
уход в ядро и обратно	~1000–5000	тысячи

- Свободный мьютекс — это ~19 тактов. Дорога конкуренция, а не мьютекс.
- Между быстрым и медленным путём — три порядка величины.

sync.Mutex в Go: паркуется горутина, а не поток

```
type Mutex struct {  
    state int32 // биты: locked | woken | starving | счётчик ждущих  
    sema  uint32 // семафор рантайма для парковки горутин  
}
```

GO

- Быстрый путь — один `CAS(state, 0, locked)`, без рантайма и без ядра.
- Спин до 4 итераций, и только если он вообще имеет смысл.
- Не вышло — горутины паркует рантайм (`gopark`), поток ОС остаётся жив.
- Поток тут же уходит крутить другие горутины: сотни нс, а не микросекунды.

Два режима sync.Mutex: normal и starvation

- **Normal**: разбуженный конкурирует с новичками — и часто проигрывает.
- Новичок уже бежит на ядре, у разбуждённого фора потеряна.
- **Starvation** (голодание): ждал дольше **1 мс** — мьютекс отдаёт строго в голову очереди.
- В этом режиме новички не спянут и встают в хвост FIFO.
- Пропускная способность чуть ниже, зато хвост задержек спасён.

Один мьютекс, четыре языка

```
std::mutex m;                                     // C++: мьютекс отдельно, данные отдельно
{ std::lock_guard lk(m); balance += class="hl-n">100; } // RAII отпустит на выходе
```

CPP

```
var mu sync.Mutex                                // Go: то же самое, дисциплина на человеке
mu.Lock(); balance += 100; mu.Unlock()
```

GO

```
let balance = Mutex::new(0i64);                  // Rust: данные ВНУТРИ мьютекса
*balance.lock().unwrap() += 100;                  // без захвата до них не добраться
```

RUST

```
with lock:                                        # Python: with вместо RAII
    balance += class="hl-n">100                    # GIL от гонок логики не спасает
```

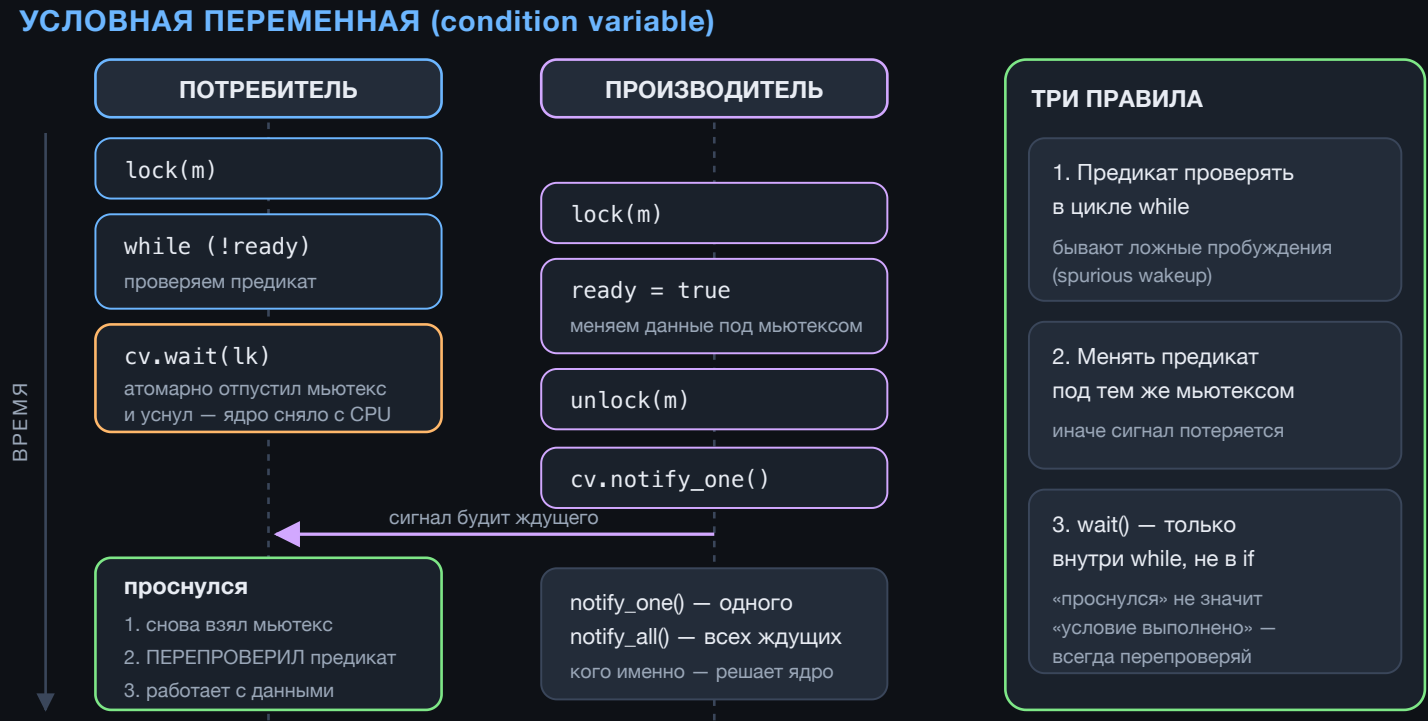
PYTHON

RWMutex: не бесплатный апгрейд

	C++	Go
тип	<code>std::shared_mutex</code> (C++17)	<code>sync.RWMutex</code>
чтение	<code>std::shared_lock lk(m);</code>	<code>mu.RLock()</code> / <code>mu.RUnlock()</code>
запись	<code>std::unique_lock lk(m);</code>	<code>mu.Lock()</code> / <code>mu.Unlock()</code>

- Помогает: много читателей и **длинные** чтения, от микросекунд.
- Вредит: на короткой секции счётчик читателей сам становится узким местом.
- `RLock` не реентерантный: рекурсивный `RLock` при ждущем писателе — дедлок.
- Правило: **сначала померь**. Замена вслепую часто даёт минус.

Condition variable: ждём условие, а не блокировку



Мьютекс отвечает «внутри кто-то есть?», condvar — «данные уже готовы?»

- **Предикат** — то условие, которого мы ждём: `ready`, «очередь не пуста».

- Щели между «отпустил мьютекс» и «уснул» нет — иначе чужой `notify` влез бы в неё.
- Поэтому `condvar` не собрать из мьютекса и `sleep`: атомарность даёт только ядро.

Condvar в коде: цикл, мьютекс, notify

```
std::unique_lock lk(m);  
cv.wait(lk, []{ return ready; });    // (1) предикат в цикле: это и есть while
```

CPP

```
{ std::lock_guard g(m); ready = true; }    // (2) предикат меняем под мьютексом  
cv.notify_one();                          // (3) notify – можно уже снаружи
```

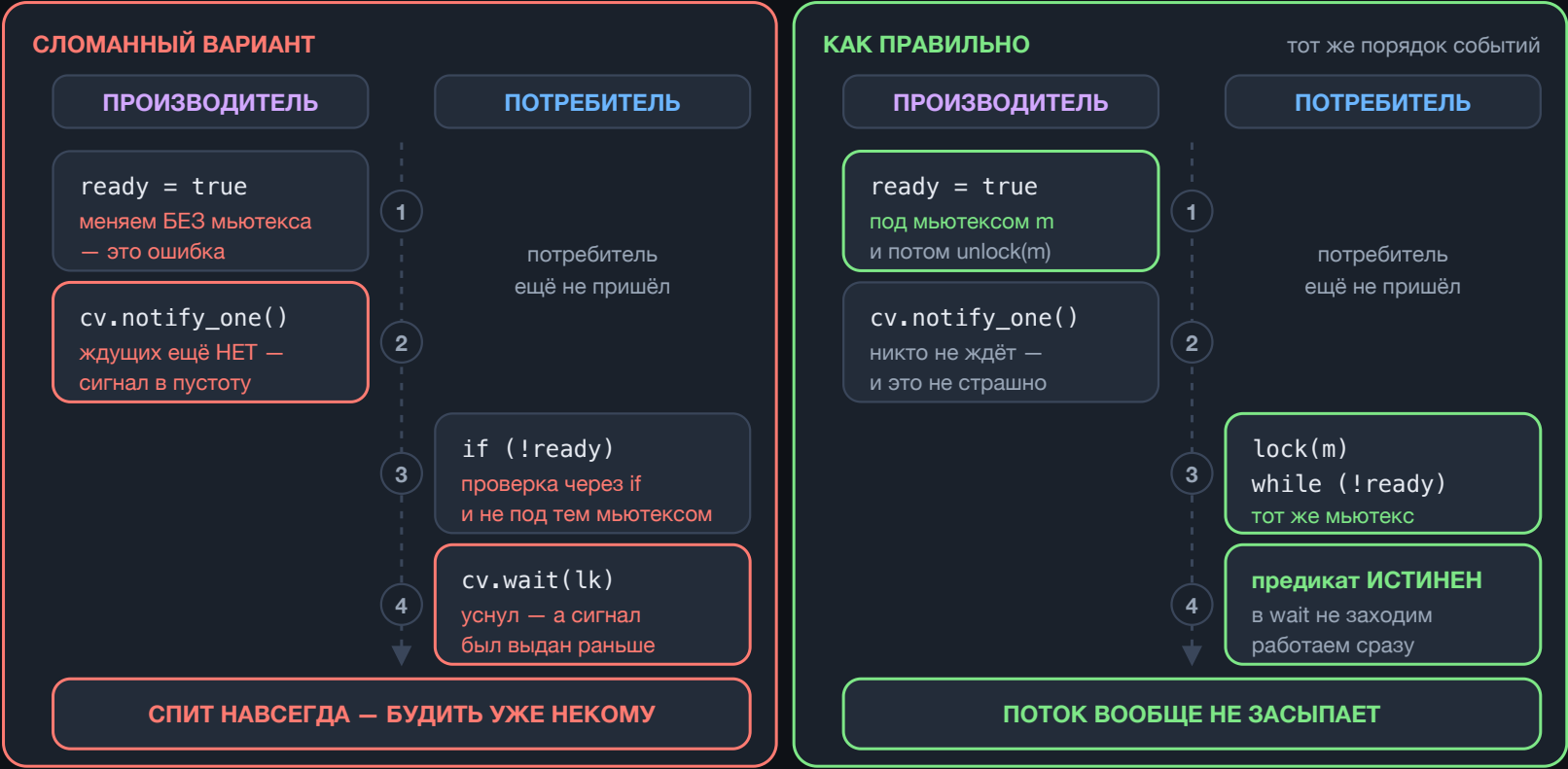
CPP

```
mu.Lock()  
for !ready {                            // никакого if: sync.Cond предикатов не умеет  
    cond.Wait()  
}  
mu.Unlock()
```

GO

Lost wakeup: сигнал в пустоту

ПОТЕРЯННОЕ ПРОБУЖДЕНИЕ (lost wakeup)



Слева баг, справа тот же порядок событий — но код корректен

- `Condvar` — не почтовый ящик: сигнал нигде не хранится и не копится.

- Спасает не удачный порядок потоков, а инвариант из правил (1) и (2).

Ограниченная очередь: мьютекс и два condvar

```
std::mutex m;  std::condition_variable not_empty, not_full;
std::deque<int> q;  const size_t cap = 1024;
void push(int v) {
    std::unique_lock lk(m);
    not_full.wait(lk, [&]{ return q.size() < cap; });
    q.push_back(v);
    lk.unlock();  not_empty.notify_one();
}
int pop() {
    std::unique_lock lk(m);
    not_empty.wait(lk, [&]{ return !q.empty(); });
    int v = q.front();  q.pop_front();
    lk.unlock();  not_full.notify_one();
    return v;
}
```

CPP

- В Go ровно это — одна строка: `make(chan int, 1024)`.

Семафор: ограничиваем параллелизм

```
std::counting_semaphore<class="hl-n">8> sem{class="hl-n">8};    // C++20, <semaphore>
sem.acquire();          // ждём разрешение
do_query();
sem.release();          // возвращаем разрешение
```

CPP

```
sem := make(chan struct{}, 8)    // буферизованный канал = семафор
for _, id := range ids {
    sem <- struct{}{}           // acquire: девятый заблокируется
    go func(id int) {
        defer func() { <-sem }() // release
        query(id)
    }(id)
}
```

GO

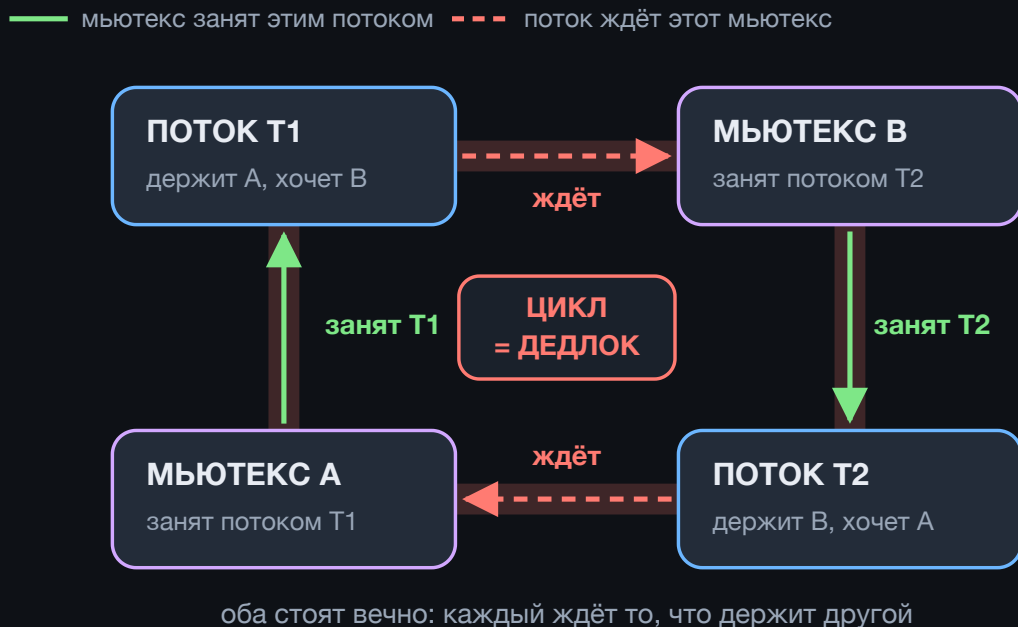
- Семафор **не имеет владельца**: захватил один поток, вернуть может другой.

Latch, barrier, WaitGroup, Once

- `std::latch` — одноразовый счётчик вниз: «стартовый выстрел» или ожидание N задач.
- `std::barrier` — многоразовый: все пришли, все пошли дальше.
- `sync.WaitGroup` — Go-шный latch: `Add`, `Done` в `defer`, `Wait` снаружи.
- В Go 1.25 есть `wg.Go(f)` — короче и труднее ошибиться.
- `sync.Once` / `std::call_once` — ленивая инициализация ровно один раз.

Дедлок: цикл в графе ожидания

ДЕДЛОК (deadlock): ЦИКЛ В ГРАФЕ ОЖИДАНИЯ



ЧЕТЫРЕ УСЛОВИЯ КОФФМАНА

1. Взаимное исключение
ресурс нельзя поделить
2. Удержание и ожидание
держу одно, жду другое
3. Невозможность отнять
отпускает только владелец
4. Круговое ожидание
тот самый цикл слева

убери любое — дедлока не будет

ЛЕКАРСТВО

захватывать блокировки всегда в одном и том же порядке
например, по адресу мьютекса или по его id — тогда круг просто не может замкнуться

Каждый ждёт того, кто ждёт его; убери любое условие — дедлока не будет

- Дедлок — не «зависло», а строго определяемое состояние.

- Есть цикл в графе «кто кого ждёт» — есть дедлок.

- Из этого растут все инструменты диагностики.

Как не попасть в дедлок

```
std::scoped_lock lk(from.m, to.m);    // C++17: берёт оба безопасно
from.balance -= amount;  to.balance += amount;
```

CPP

```
first, second := &from.mu, &to.mu    // Go: порядок задаём руками, по id
if from.id > to.id { first, second = second, first }
first.Lock(); defer first.Unlock()
second.Lock(); defer second.Unlock()
```

GO

- **Глобальный порядок захвата** — главное лекарство. Равных сортируйте по id.
- Таймауты не лечат, но превращают вечное зависание в видимую ошибку.
- **Не вызывайте чужой код под мьютексом**: колбэк, виртуальный метод, плагин.

Livelock, голодание, инверсия приоритетов

- **Livelock**: все бегает, никто не продвигается. Двое в коридоре шагают синхронно.
- **Голодание (starvation)**: поток вечно проигрывает гонку за блокировку.
- **Инверсия приоритетов**: низкий держит замок, средний вытесняет низкого, высокий не двигается вообще.
- **Mars Pathfinder, 1997**: марсоход перезагружался сам — это была она.
- **Lock convoy**: все выстроились в очередь, симптом — CPU в system time.

Как Go ловит дедлок — и чего не ловит

```
fatal error: all goroutines are asleep - deadlock!
```

- Рантайм ругается, только когда спят **ВСЕ** горутины.
- Одна крутящаяся `for {}` или живой таймер — и рантайм молчит.
- Частичный дедлок (2 горутины из 10 000) не ловится вообще.
- Следите за `runtime.NumGoroutine()` в метриках: это ваш детектор.
- Дамп стеков: `kill -QUIT <pid>` или Ctrl-\\, плюс `GOTRACEBACK=all`.

Итог части 3

- Мьютекс — слово в памяти плюс договор с ядром на случай драки.
- Свободный захват ~4 нс, уход в ядро — микросекунды. Три порядка.
- Дорога конкуренция, а не сам примитив: линия переезжает между ядрами.
- Condvar: предикат в цикле, менять под мьютексом, notify — можно снаружи.
- Семафор помнит сигналы, condvar — нет. В этом вся разница.
- Дедлок — цикл в графе ожидания. Ломайте круговое ожидание порядком.

4. CSP: не делите память — общайтесь

- Канал: очередь и точка встречи в одном объекте
- `select` — ждать любое событие, `context` — отменять работу
- Паттерны: пул воркеров, конвейер, семафор
- Утечки горутин и честная цена канала в наносекундах
- То же самое в C++, Rust, Python, Java, Erlang

Общее состояние никуда не делось. Просто теперь оно ездит по проводам, а не лежит под мьютексом.

Идея CSP: у данных один владелец

- **CSP** (Communicating Sequential Processes) — Тони Хоар, 1978
- Независимые процессы, общей памяти нет, только сообщения
- У данных ровно **один владелец**; владение уезжает с сообщением
- Двух владельцев не бывает — значит, гонки данных (data race) нет
- Девиз Go: «делитесь памятью, общаясь», а не наоборот
- Оговорка: для счётчика или флага мьютекс проще и дешевле

Два вида каналов

ДВА ВИДА КАНАЛОВ



ПАМЯТКА

чтение из закрытого канала — сразу ноль, без блокировки
`v, ok := <-ch`

запись в закрытый — **ПАНИКА**
повторный `close()` — тоже паника

закрывает всегда **писатель**
читатель не знает, кто ещё пишет

`nil`-канал блокирует навсегда

Небуферизованный канал — рандеву, буферизованный — очередь на сам сообщений

Первая программа: отправили и приняли

```
ch := make(chan int)           // канал без буфера

go func() {
    ch <- heavy(42)             // посчитали и отдали
}()

v := <-ch                      // здесь ждём, пока не придёт
fmt.Println(v)
```

GO

- Ни мьютекса, ни флага: читатель получает готовое значение
- Отправили — значение больше не наше, трогать его нельзя

Одна идея, три языка

```
ch := make(chan int) // Go: канал в языке
go func() { ch <- heavy(42) }()
v := <-ch
```

GO

```
let (tx, rx) = mpsc::channel(); // Rust: канал в библиотеке
thread::spawn(move || {
    tx.send(heavy(42)).unwrap();
});
let v = rx.recv().unwrap();
```

RUST

```
std::promise<int> p; // C++: канала в стандарте нет
auto f = p.get_future();
std::jthread t([&p] { p.set_value(heavy(class="hl-n">42)); });
int v = f.get();
```

CPP

range и close: конец данных и сигнал всем

G0

```
func producer(out chan<- int) {  
    defer close(out)           // закрывает тот, кто пишет  
    for i := 0; i < 5; i++ { out <- i }  
}  
  
for v := range ch { use(v) }   // цикл сам кончится на close(ch)  
  
done := make(chan struct{})    // сигнал сразу всем  
close(done)                    // проснулись все, кто ждал done
```

- `close` — сигнал «данных больше не будет», а не освобождение памяти
- `chan<- T` и `<-chan T` в сигнатуре — страховка от компилятора

select: ждём любое из событий

```
select {  
  case v := <-in:  
    process(v)  
  case out <- result:           // готовность ОТПРАВИТЬ — тоже событие  
  case <-done:  
    return  
}
```

GO

- Готовы несколько веток — рантайм выбирает **случайную**
- Не готова ни одна — горутина засыпает сразу на всех
- `ch = nil` выключает ветку: `nil`-канал блокируется вечно

select: таймаут и попытка без блокировки

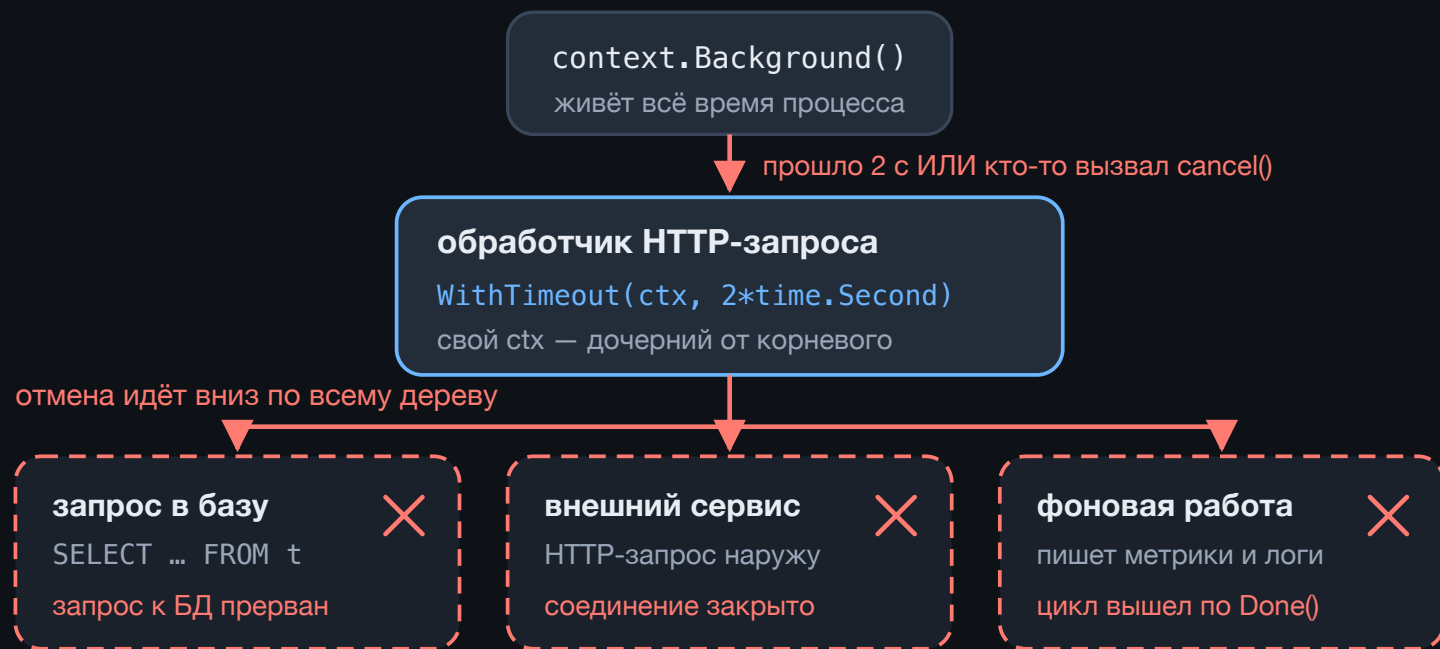
```
select {  
case v := <-in:  
    return v, nil  
case <-time.After(200 * time.Millisecond):  
    return 0, errTimeout           // таймаут — обычная ветка  
}  
  
select {  
case ch <- v:                     // отправили, если взяли  
default:                         // иначе не ждём ни такта  
}
```

GO

- `default` превращает `select` в неблокирующую попытку
- В горячем цикле вместо `time.After` — `NewTimer` плюс `Stop`

Отмена: дерево контекстов

ДЕРЕВО КОНТЕКСТОВ И ОТМЕНА



ctx ничего не останавливает силой — он только сообщает «пора заканчивать»
выйти из работы обязана сама функция: проверяет ctx.Done() и возвращается

ПРАВИЛА

ctx — первым аргументом функции

defer cancel()
обязательно, иначе течёт

ctx.Done()
канал, который закрывается при отмене

В C++ — то же самое

std::jthread + stop_token
просьба остановиться,
а не убийство потока

Отмена спускается от корня ко всем дочерним задачам сразу

context в коде: таймаут на один запрос

```
func fetch(ctx context.Context, url string) ([]byte, error) {  
    ctx, cancel := context.WithTimeout(ctx, 2*time.Second)  
    defer cancel() // отпустить таймер и узел  
  
    req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)  
    resp, err := http.DefaultClient.Do(req) // оборвётся по ctx  
    if err != nil { return nil, err }  
    defer resp.Body.Close()  
    return io.ReadAll(resp.Body)  
}
```

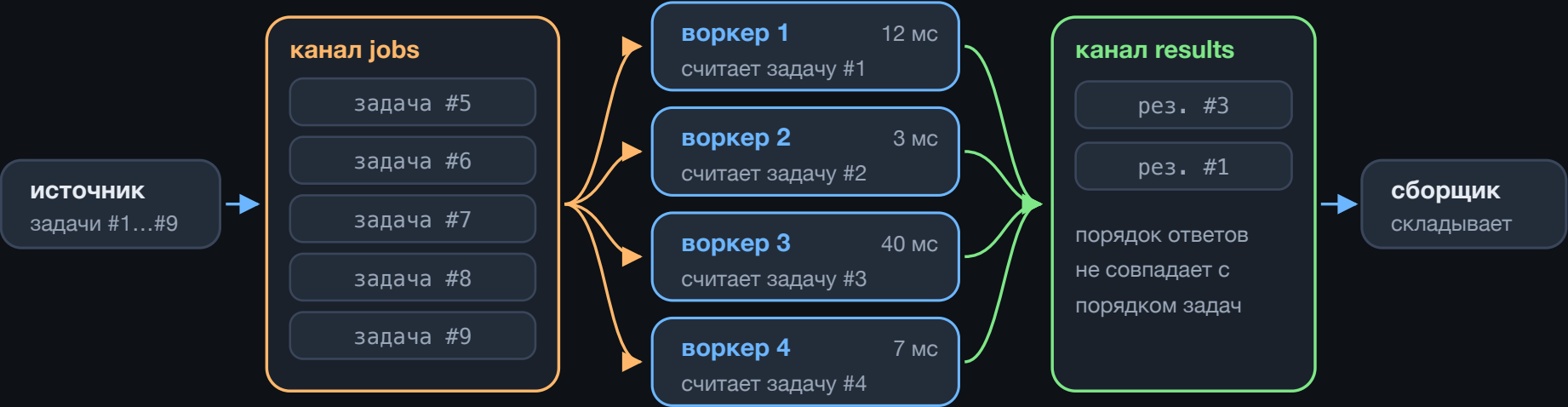
GO

- `WithValue` — только для метаданных запроса, не для параметров функции
- `ctx.Err()`: `Canceled` — «нас попросили», `DeadlineExceeded` — «мы не успели»

Паттерн: пул воркеров

ПУЛ ВОРКЕРОВ (worker pool)

4 горутины-воркера разбирают общий поток задач; задачи разной длительности



все воркеры тянут из ОДНОГО канала: кто освободился — тот и взял следующую

счётная работа воркеров ≈ по числу ядер больше — только лишние переключения	ждём сеть или диск воркеров нужно намного больше они почти всё время просто стоят	кто закрывает канал всегда тот, кто в него пишет читатель увидит конец по range
--	--	--

Фиксированное число воркеров разбирает общую очередь задач

Пул воркеров: код

GO

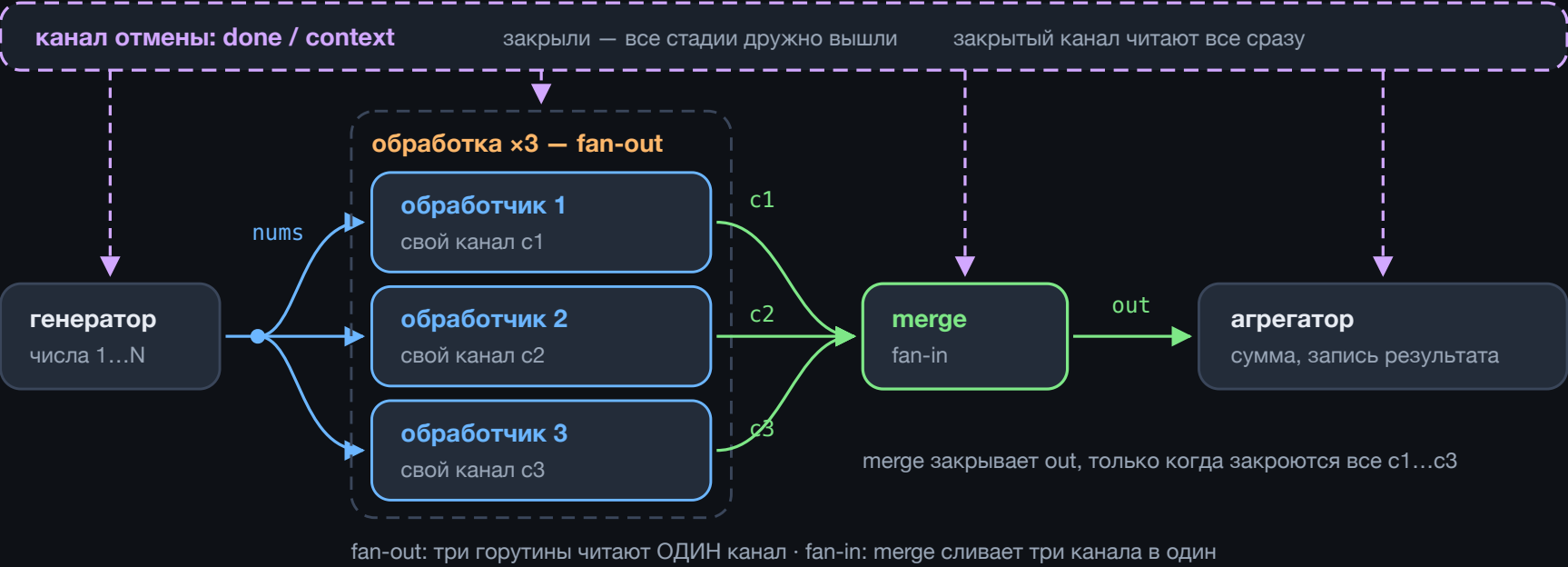
```
jobs := make(chan Job)
results := make(chan Result)

var wg sync.WaitGroup
for w := 0; w < runtime.GOMAXPROCS(0); w++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for j := range jobs { results <- do(j) } // пул жив, пока jobs открыт
    }()
}
go func() { wg.Wait(); close(results) }() // пишем — мы и закрываем
```

- Забыли `close(jobs)` — воркеры висят в `range`, программа не кончается

Конвейер: fan-out и fan-in

КОНВЕЙЕР: FAN-OUT И FAN-IN



УТЕЧКА ГОРУТИНЫ (goroutine leak)

если из канала никто не читает, горутина-писатель зависнет на отправке навсегда
сборщик мусора её не уберёт: она живая — просто вечно ждёт. Спасает как раз канал отмены

Один канал читают несколько обработчиков, merge сливает результаты обратно

fan-in: пишем merge

GO

```
func merge(cs ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    wg.Add(len(cs))
    for _, c := range cs {
        go func() {
            defer wg.Done()
            for v := range c { out <- v }      // сливаем всё в один канал
        }()
    }
    go func() { wg.Wait(); close(out) }()
    return out
}
```

- Писателей несколько — закрывает отдельная горутина после `wg.Wait()`

Семафор: ограничить одновременность

```
sem := make(chan struct{}, 8)           // Go: восемь пропусков
sem <- struct{}{}                       // взять (ждёт, если мест нет)
defer func() { <-sem }()                // вернуть
```

GO

```
std::counting_semaphore<class="h1-n">8> sem{class="h1-n">8};    // C++20
sem.acquire(); work(); sem.release();
```

CPP

```
sem = threading.Semaphore(class="h1-n">8)           # Python
with sem: work()
```

PYTHON

errgroup: ждать всех и поймать первую ошибку

```
g, ctx := errgroup.WithContext(ctx)
g.SetLimit(8) // тот же семафор, но внутри
for _, u := range urls {
    g.Go(func() error {
        return fetch(ctx, u) // первая ошибка отменит ctx
    })
}
if err := g.Wait(); err != nil { // ждём всех, возвращаем первую ошибку
    return err
}
```

GO

- Закрывает большинство задач «сделай N дел сразу и скажи, если сломалось»

Утечки горутин: кто её остановит?

1. **Писатель без читателя:** `ch <- v`, а читатель ушёл по таймауту
 2. **Читатель без писателя:** `range ch`, а `close` никто не сделал
 3. **Забытая отмена:** фоновый цикл не смотрит в `ctx.Done()`
- Цена: стек горутины 2 КБ плюс всё, на что она ссылается
 - Ловим: `runtime.NumGoroutine()`, `/debug/pprof/goroutine`, `go leak` в тестах

Цена канала: почему он дороже мьютекса

задача	инструмент	порядок цены
счётчик, флаг	<code>sync/atomic</code>	около 2 нс
защитить структуру	<code>sync.Mutex</code>	около 4 нс без конкуренции
передать владение	канал	60–120 нс на сообщение
дождаться N задач	<code>sync.WaitGroup</code>	почти бесплатно

- Внутри канала — мьютекс, кольцевой буфер и очереди ждущих горутин
- Горячий счётчик через канал — в десятки раз дороже атомика

Шесть языков — шесть философий

язык	как принято	чем платим
Go	горутины, каналы, <code>context</code>	утечки горутин, цена канала
Rust	каналы плюс <code>Send</code> / <code>Sync</code>	спор с компилятором вместо отладки
Java	пулы, <code>CompletableFuture</code> , виртуальные потоки	многословность
Python	GIL: потоки для IO, процессы для счёта	без GIL только с 3.13
JS	один поток, event loop, Web Workers	общей памяти почти нет
Erlang	акторы, shared-nothing, супервизоры	копирование сообщений

- Rust ловит гонки на компиляции: тип без `Send` не уедет в поток

Актор против CSP

- В CSP адресат — **канал**: кто прочитает, отправитель не знает
- В акторах адресат — **процесс** со своим почтовым ящиком
- Отправка актору асинхронна, не блокирует, сообщение копируется
- Ящик без предела растёт, пока не кончится память
- Канал с ёмкостью даёт обратное давление (backpressure) бесплатно
- Актор в Go собирается руками: структура с каналом и одна горутина

Мини-итог части

- Канал = очередь плюс точка синхронизации; передаёт **владение**
- Закрывает писатель, и только один; `close` — сигнал сразу всем
- `select` даёт таймауты, `context` — отмену всему дереву задач
- Паттерны: пул воркеров, конвейер с fan-out/fan-in, семафор
- У каждой горутины известен тот, кто её остановит
- Канал стоит около 100 нс: мьютекс, копия и переезд кеш-линии

5. Атомарные операции и lock-free

- Что такое атомарность и чем атомик отличается от `volatile`
- Базовый набор: `load`, `store`, `exchange`, `fetch_add`, `CAS`
- Реальная цена: 1.8 нс в одиночку и 17 нс под конкуренцией
- Spinlock: почему в прикладном коде его быть не должно
- Гарантии прогресса: lock-free и wait-free — это не про скорость
- Стек Трайбера, ABA и главная боль — освобождение памяти

Атомарная операция: неделимая для наблюдателей

- Для чужого потока операция **либо не началась, либо уже закончилась**
- Промежуточного состояния не существует: «половину» увидеть нельзя
- «Одна строка» не равно «одна операция»
- `x++` — это три шага: прочитать, прибавить, записать
- Три шага — три момента вклиниться. Это и есть гонка данных (data race)
- Атомарность отвечает за **одну** операцию, а не за порядок разных

```
int          plain = 0;    plain++;    // load, add, store – гонка
std::atomic<int> safe{0};  safe++;    // одна неделимая операция
```

CPP

Атомик в C++, Go и соседях

Задача	C++	Go	Java
счётчик	<code>std::atomic<int64_t></code>	<code>atomic.Int64</code>	<code>AtomicLong</code>
флаг	<code>std::atomic<bool></code>	<code>atomic.Bool</code>	<code>AtomicBoolean</code>
указатель	<code>std::atomic<T*></code>	<code>atomic.Pointer[T]</code>	<code>AtomicReference</code>

- C++: `std::atomic<T>` для тривиально копируемых `T`
- `is_always_lock_free` — на компиляции, `is_lock_free()` — в рантайме
- `std::atomic_ref<T>` (C++20) — атомарный доступ к обычной переменной
- Go: пакет `sync/atomic`; удобные типы появились в Go 1.19
- Rust — `AtomicI64` и друзья; в Python атомиков нет, там свои правила

`volatile` — это не про потоки

Язык	Что на самом деле значит <code>volatile</code>
C, C++	«не оптимизируй этот доступ» — для регистров железа
Java, C#	атомарность плюс барьер памяти — это ближе к <code>atomic</code>
Go, Rust	такого слова нет вообще: только <code>sync/atomic</code> и <code>Atomic*</code>

- В C++ `volatile` не даёт ни атомарности, ни порядка между потоками
- Гонка с `volatile` остаётся гонкой — и остаётся UB
- Одно слово, три разных смысла: отсюда вечная путаница

Базовый набор: из чего строится всё

- `load` / `store` — прочитать или записать значение целиком
- `exchange` — записать новое, вернуть старое; это и есть test-and-set
- `fetch_add` / `fetch_sub` / `fetch_or` — read-modify-write за одну операцию
- `compare_exchange` (CAS) — «если лежит ожидаемое, положи новое»
- CAS универсален: из него собирается max, вставка в список, что угодно

```
std::atomic<int64_t> n{class="hl-n">0};  
int64_t was = n.fetch_add(class="hl-n">1);    // вернёт СТАРОЕ значение: 0
```

CPP

```
var n atomic.Int64  
now := n.Add(1)                // вернёт НОВОЕ значение: 1
```

GO

Одна инструкция или цикл: x86 и ARM

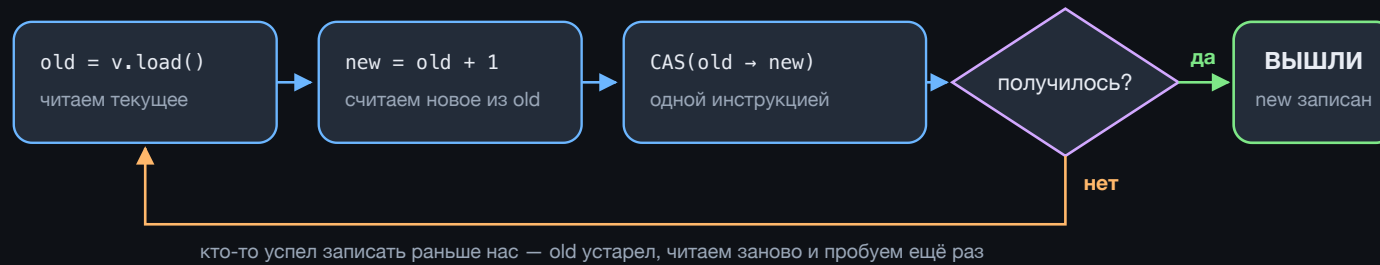
ASM

```
; x86-64: префикс LOCK делает операцию атомарной
    lock xadd    [rdi], rax    ; fetch_add
    lock cmpxchg [rdi], rcx    ; CAS, ожидаемое в rax
; ARM64 до ARMv8.1: пара LL/SC
1:    ldxr    x1, [x0]          ; load-exclusive
    add     x2, x1, #1
    stxr    w3, x2, [x0]        ; store-exclusive, w3==0 => успех
    cbnz    w3, 1b             ; не вышло — крутим заново
; ARM64 LSE (ARMv8.1+, все Apple Silicon): снова одна инструкция
    ldadd    x1, x2, [x0]        ; атомарное сложение
    casal    x1, x2, [x0]        ; CAS с acquire-release
```

- `fetch_add` — одна аппаратная операция, а CAS-луп может крутиться
- LL/SC: «загрузи эксклюзивно, запиши условно, не вышло — повтори»

CAS: сравни и поменяй

ЦИКЛ CAS compare-and-swap: «сравни со старым и запиши новое» — атомарно, одной операцией



C++

```
std::atomic<int> v{0};
int old = v.load();
while (!v.compare_exchange_weak(old, old+1))
{ }
```

при провале CAS сам кладёт в old
актуальное значение — пересчитывать не надо

Go

```
var v atomic.Int64
for {
    old := v.Load()
    if v.CompareAndSwap(old, old+1) {
        break
    }
}
```

здесь old пересчитываем сами — Load()

Почему CAS всегда пишут в цикле

weak-версия может провалиться «просто так», даже если значение никто не менял: на ARM это пара ldxr/stxr — «загрузи эксклюзивно / запиши условно», и эксклюзивность сбивает прерывание, миграция потока, запись соседа рядом.

Цикл CAS: читаем, считаем новое, пробуем записать — при провале начинаем заново

- Это **ОПТИМИСТИЧНЫЙ** подход: конфликт не предотвращаем, а переделываем работу

- В цикле — `_weak`; `_strong` прячет цикл внутри и нужен для одиночной попытки

Из CAS строим что угодно: атомарный максимум

```
void atomic_max(std::atomic<long>& m, long v) {  
    long cur = m.load(std::memory_order_relaxed);  
    while (v > cur && !m.compare_exchange_weak(cur, v)) {}  
}
```

CPP

```
func atomicMax(m *atomic.Int64, v int64) {  
    for {  
        cur := m.Load()  
        if v <= cur || m.CompareAndSwap(cur, v) {  
            return  
        }  
    }  
}
```

GO

- Аппаратного «атомарного максимума» нет — собираем сами
- Форма любого алгоритма без блокировок (lock-free): прочитал, посчитал, опубликовал

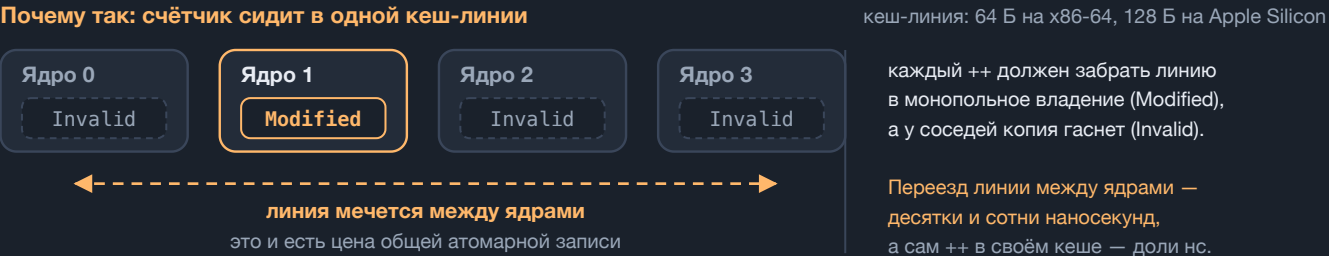
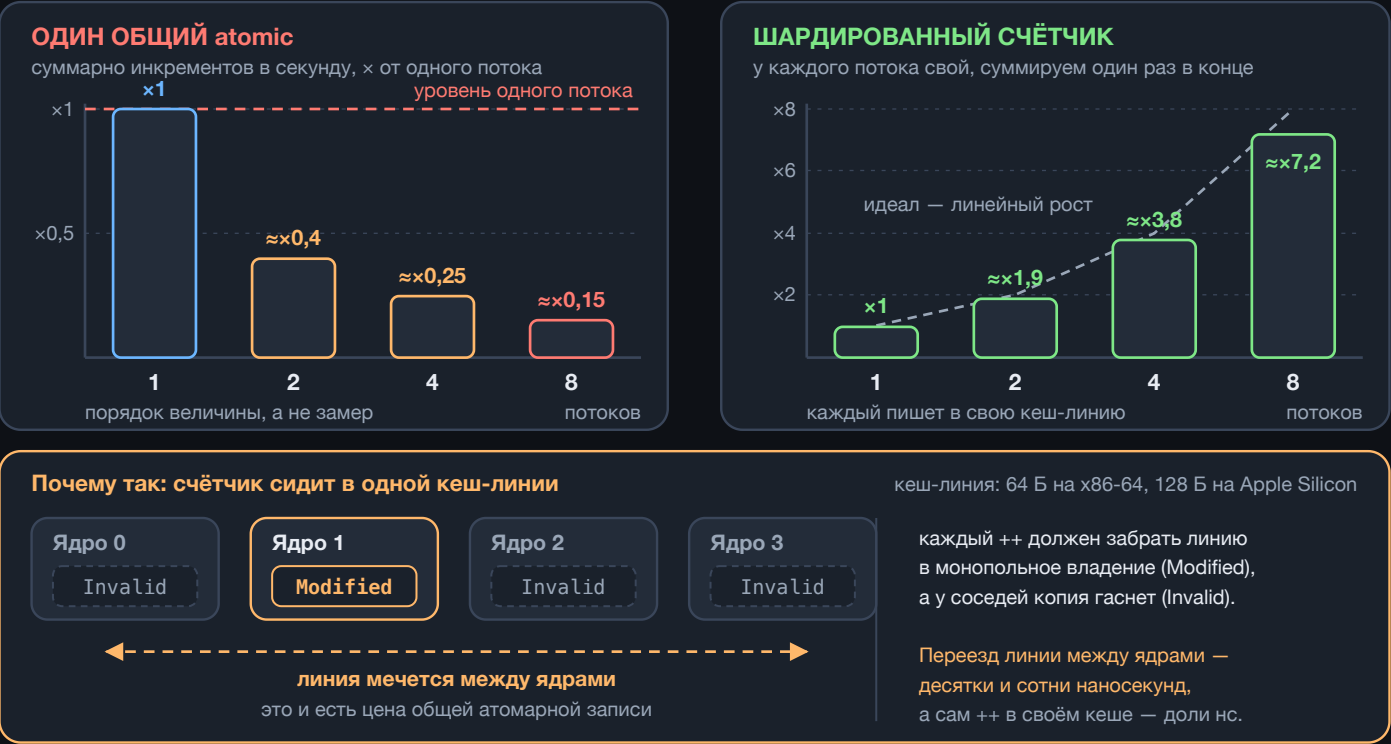
Сколько стоит атомик без конкуренции

Операция (M4, один поток)	нс	тактов
обычный <code>++</code> в L1	0.33	1.4
<code>fetch_add</code> , relaxed	1.79	7.9
<code>fetch_add</code> , seq_cst	1.78	7.8
CAS-цикл	1.82	8.0
<code>pthread_mutex</code> lock + unlock	4.25	18.7

- Атомик дороже обычной записи примерно в 5 раз. Это дёшево
- `relaxed` и `seq_cst` стоят одинаково: модель памяти — про корректность
- Мьютекс без конкуренции — 4 нс, а не 75 нс из старых табличек

Цена конкуренции: больше потоков — медленнее

АТОМАРНЫЙ СЧЁТЧИК НЕ МАСШТАБИРУЕТСЯ атомарность честная, но переменная одна на всех



Один общий счётчик деградирует с числом потоков, шардированный — растёт

- Замер на M4: 2 потока — 3.6 нс, 4 — 10.9, 8 — 16.6

- Разнесли счётчики на 128 байт — 0.46 нс. Разница в 36 раз

Spinlock: наивный TAS

```
class TASLock {                                // test-and-set: как НЕ надо
    std::atomic<bool> locked_{false};
public:
    void lock() {
        // exchange – это ЗАПИСЬ, и она повторяется на каждой итерации
        while (locked_.exchange(true, std::memory_order_acquire)) {}
    }
    void unlock() {
        locked_.store(false, std::memory_order_release);
    }
};
```

CPP

- Каждая попытка — запись, то есть запрос на монопольное владение линией
- Десять ждущих ядер гоняют одну линию по кругу
- Владельцу нужна та же линия, чтобы отпустить лок

TTAS и backoff: как это лечат

```
void lock() {                                     // test-and-test-and-set
    for (int backoff = 1;; backoff = std::min(backoff * 2, h
        if (!locked_.exchange(true, std::memory_order_acquire))
            return;
        // ждём ОБЫЧНЫМ чтением: линия остаётся Shared, трафика нет
        while (locked_.load(std::memory_order_relaxed))
            for (int i = 0; i < backoff; ++i) cpu_pause();
    }
}
```

CPP

- Ждём чтением, пишем только когда есть шанс
- Пауза растёт экспоненциально, потом честно уступаем планировщику
- В C++20 это уже готово: `atomic<T>::wait()` и `notify_one()`

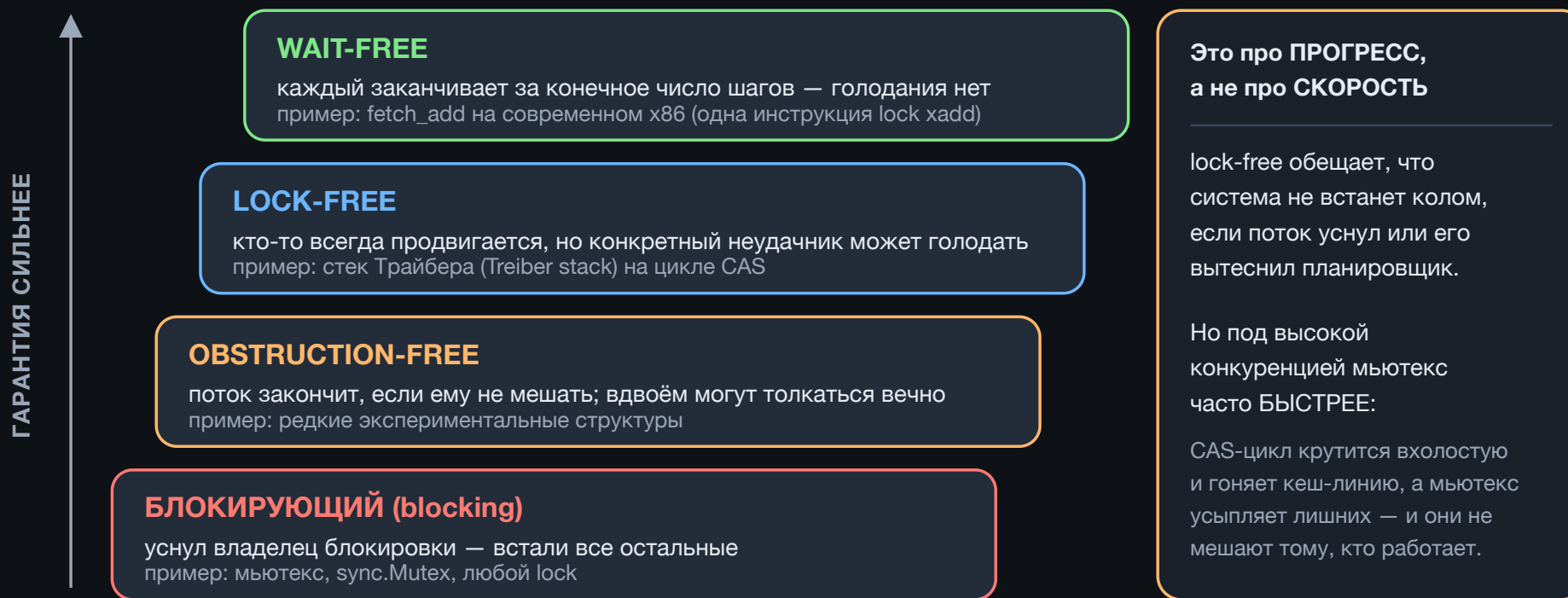
Spinlock: когда можно, а когда категорически нет

- **Можно**, если выполнены сразу все условия:
 - код ядра ОС или обработчик прерывания, где спать нельзя
 - критическая секция — единицы-десятки наносекунд
 - потоки прибиты к ядрам, свободных ядер хватает на всех
- **Нельзя** в прикладном коде поверх вытесняющего планировщика
- Владельца вытеснили посреди секции — остальные жгут кванты впустую
- В Go спиннер занимает слот `P`: прогресса может не быть совсем
- `std::mutex` и `sync.Mutex` спилят сами, а потом честно засыпают

Гарантии прогресса: lock-free и wait-free

ЛЕСТНИЦА ГАРАНТИЙ ПРОГРЕССА

что обещает алгоритм, если один поток внезапно остановился



Лестница гарантий: блокирующий, obstruction-free, lock-free, wait-free

- Где гарантия реально нужна: реалтайм, обработка звука, ядро ОС, трейдинг

- В обработчике сигнала мьютекс нельзя в принципе — остаётся только атомик

Стек Трайбера: lock-free на одном CAS

```
struct Node { int value; Node* next; };
std::atomic<Node*> head{nullptr};

void push(int v) {                                     // CAS сам обновит n->next
    Node* n = new Node{v, head.load(std::memory_order_relaxed)};
    while (!head.compare_exchange_weak(n->next, n)) {}
}

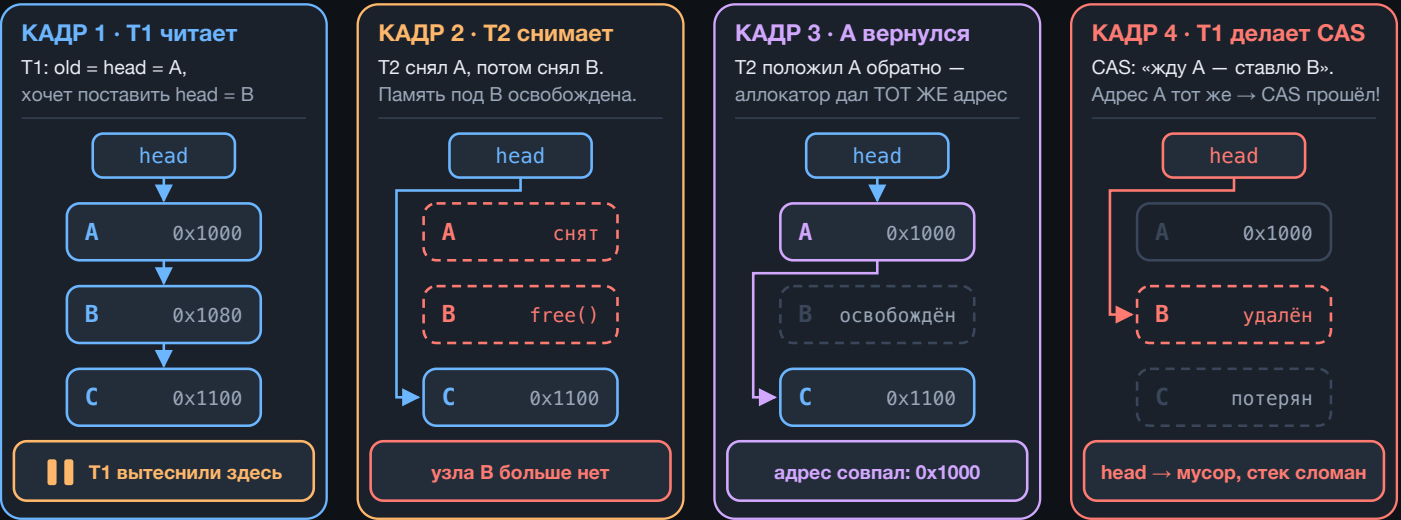
bool pop(int& out) {
    Node* old = head.load(std::memory_order_acquire);
    while (old && !head.compare_exchange_weak(old, old->next)) {}
    if (!old) return false;
    out = old->value;
    delete old;                                       // <- вот здесь настоящая боль
    return true;
}
```

CPP

ABA: тот же адрес, другой объект

ПРОБЛЕМА ABA

CAS сравнивает адрес, а не историю: указатель ушёл и вернулся — и CAS ничего не заметил



ЛЕКАРСТВА

не дать адресу переиспользоваться, пока кто-то держит на него старый указатель

версия у указателя

CAS сравнивает пару {адрес, версия}, а версия только растёт

hazard pointers

поток объявляет, на какой узел смотрит — такой не освобождают

epoch-based

освобождаем пачкой, когда все потоки вышли из эпохи

RCU

read-copy-update: читатели не ждут, старая копия живёт

GC — Go, Java

пока на узел кто-то смотрит, адрес не отдадут заново

ABA: T1 уснул, стек успел измениться и вернуться — CAS ничего не заметил

- Под GC (Go, Java) защищены указатели — но не индексы и не счётчики поколений

- Баг недетерминированный: «у меня всё работает» здесь не доказывает ничего

Освобождение памяти — настоящая боль lock-free

- Алгоритм на CAS пишется за вечер; «когда звать `free`?» не решается никак
- Узел нельзя удалять, пока хоть кто-то **может** держать на него указатель
- А спросить в lock-free структуре не у кого: списка читателей нет
- **hazard pointers** — поток публикует, на что смотрит (в C++26 в стандарте)
- **epoch-based** и **RCU** — чистим пачкой, когда все прошли эпоху
- **GC** — в Go и Java проблема исчезает, ценой рантайма и пауз

Берите готовое, не пишите своё

- Правило номер один: **свою lock-free структуру вы писать не должны**
- Сначала мьютекс, потом замер, и только потом — готовая библиотека
- C++: `folly`, `oneTBB`, `moodycamel::ConcurrentQueue`, `boost::lockfree`
- Rust: `crossbeam` (каналы, `deque`, `epoch`), `dashmap`
- Go: каналы, `sync.Map`, `sync.Pool`, `atomic.Pointer[T]`
- Java: `ConcurrentHashMap`, `LongAdder` — шардированный счётчик из коробки

sync.Map: ровно два сценария

- Ключ пишется один раз и читается много — read-mostly кеш
- Разные горутины работают с **непересекающимися** наборами ключей
- Во всех остальных случаях `map` под `RWMutex` быстрее и типобезопаснее
- С Go 1.24 внутри HashTrieMap: стало лучше, но правило не изменилось

Copy-on-write: публикуем указатель

```
var cfg atomic.Pointer[Config] // Go 1.19+, типобезопасно

func Get() *Config { return cfg.Load() } // читатели бесплатны
func Reload(c *Config) { cfg.Store(c) } // публикуем новую копию

func AddHost(h string) { // правка поля – через CAS
    for {
        old := cfg.Load()
        cp := *old // копия структуры
        cp.Hosts = append(append([]string{}, old.Hosts...), h)
        if cfg.CompareAndSwap(old, &cp) { return }
    }
}
```

GO

- Железное правило: опубликованную структуру **больше не меняем**
- В C++ то же самое: `std::atomic<std::shared_ptr<T>>` (C++20)

Итог части

- Атомик — не «быстрый мьютекс», а инструмент для одной ячейки памяти
- Без конкуренции он дешёв: 1.8 нс против 0.33 нс у обычной записи
- Под конкуренцией всё решает переезд кеш-линии: ~37 нс за миграцию
- `compare_exchange_weak` всегда живёт в цикле, `expected` обновляется сам
- Spinlock в прикладном коде — почти всегда ошибка
- Lock-free не значит быстрее; главная сложность не CAS, а `free`

Дальше: почему процессор и компилятор вообще имеют право переставлять ваши операции — и что происходит с кеш-линией, когда за неё дерутся восемь ядер.

6. Модель памяти: почему код не выполняется как написан

«Я написал `data = 42; flag = true;`. Как второй поток увидел `flag == true`, но `data == 0`?»

- Атомик делает операцию **неделимой**. Это только половина дела.
- Вторая половина: **в каком порядке** её увидят другие потоки.
- Порядок, который вы написали, — пожелание, а не приказ.

Три редактора вашего кода

Между исходником и реальной памятью сидят три независимых оптимизатора:

1. **Компилятор** — переставляет, сливает, выкидывает, держит в регистрах.
 2. **Подсистема памяти** — буфер записи (store buffer), кеши, очереди.
 3. **Процессор** — внеочередное исполнение (out-of-order), спекуляция.
- Каждый обещает ровно одно: **однопоточная** программа не заметит подмены.
 - Формально это правило «как если бы» (as-if rule). Многопоточная — заметит.

Редактор №1: компилятор поднимает чтение из цикла

```
bool flag = false; int data = 0;

void producer() { data = 42; flag = true; }

void consumer() {
    while (!flag) { }           // ждём сигнала
    std::cout << data;          // ожидаем 42
}
```

CPP

- На -00 «работает». На -02 компилятор читает `flag` один раз в регистр — и получается `while (true) {}`.

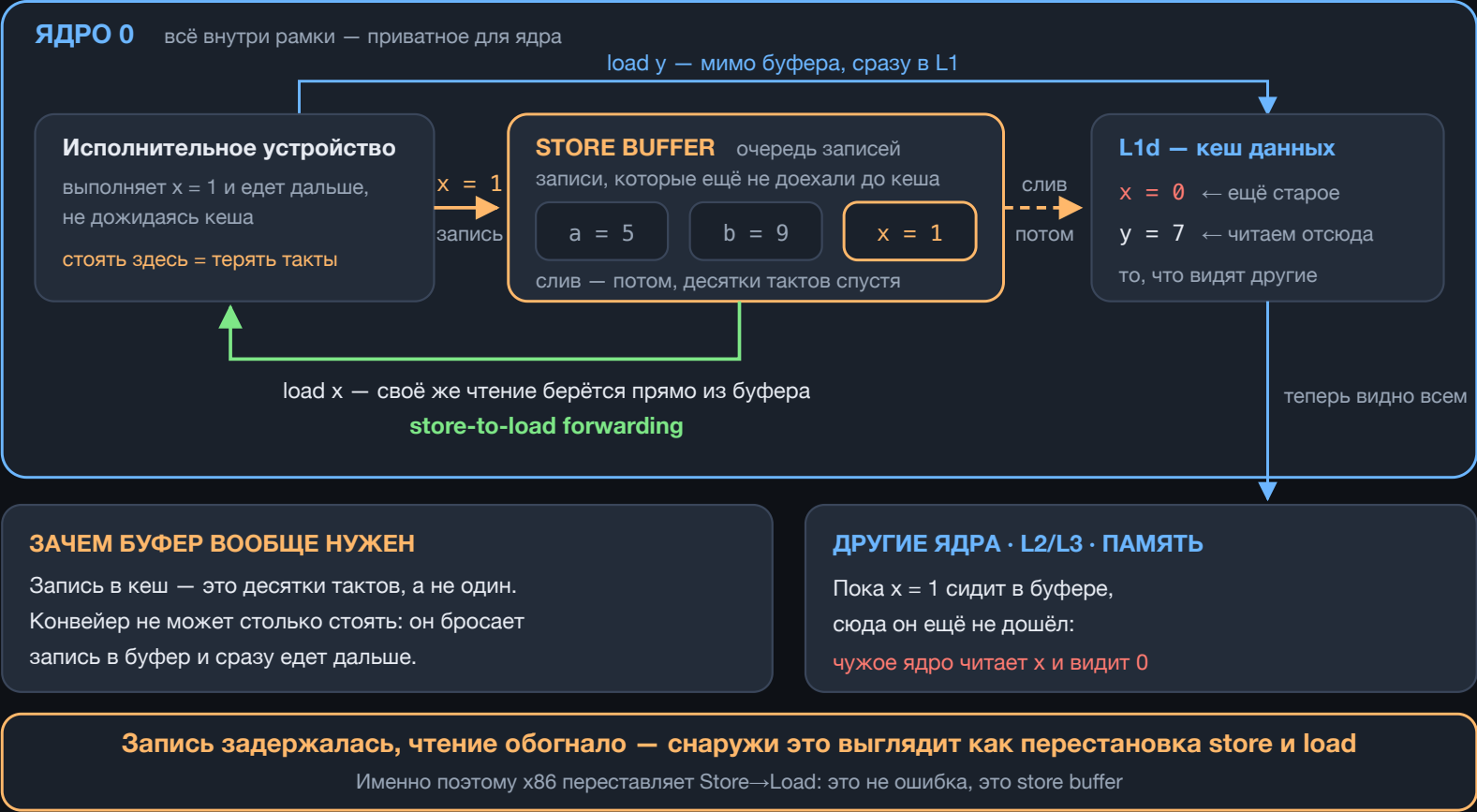
`volatile` — слово-ловушка

Язык	Что даёт <code>volatile</code>
C / C++	запрет оптимизаций доступа — и всё
Java (с JSR-133)	часть модели памяти: даёт и порядок, и видимость
C#	ослабленный порядок на чтении и записи
Go, Rust, Python	такого слова нет вовсе — есть атомарные типы

- В C++ не даёт ни атомарности, ни барьеров, ни видимости.
- Его область — регистры устройств (MMIO), обработчики сигналов, `setjmp`.
- Для потоков в C++ инструмент ровно один: `std::atomic`.

Редактор №2: буфер записи (store buffer)

STORE BUFFER: ПОЧЕМУ ЗАПИСЬ ОПАЗДЫВАЕТ, А ЧТЕНИЕ ОБГОНЯЕТ

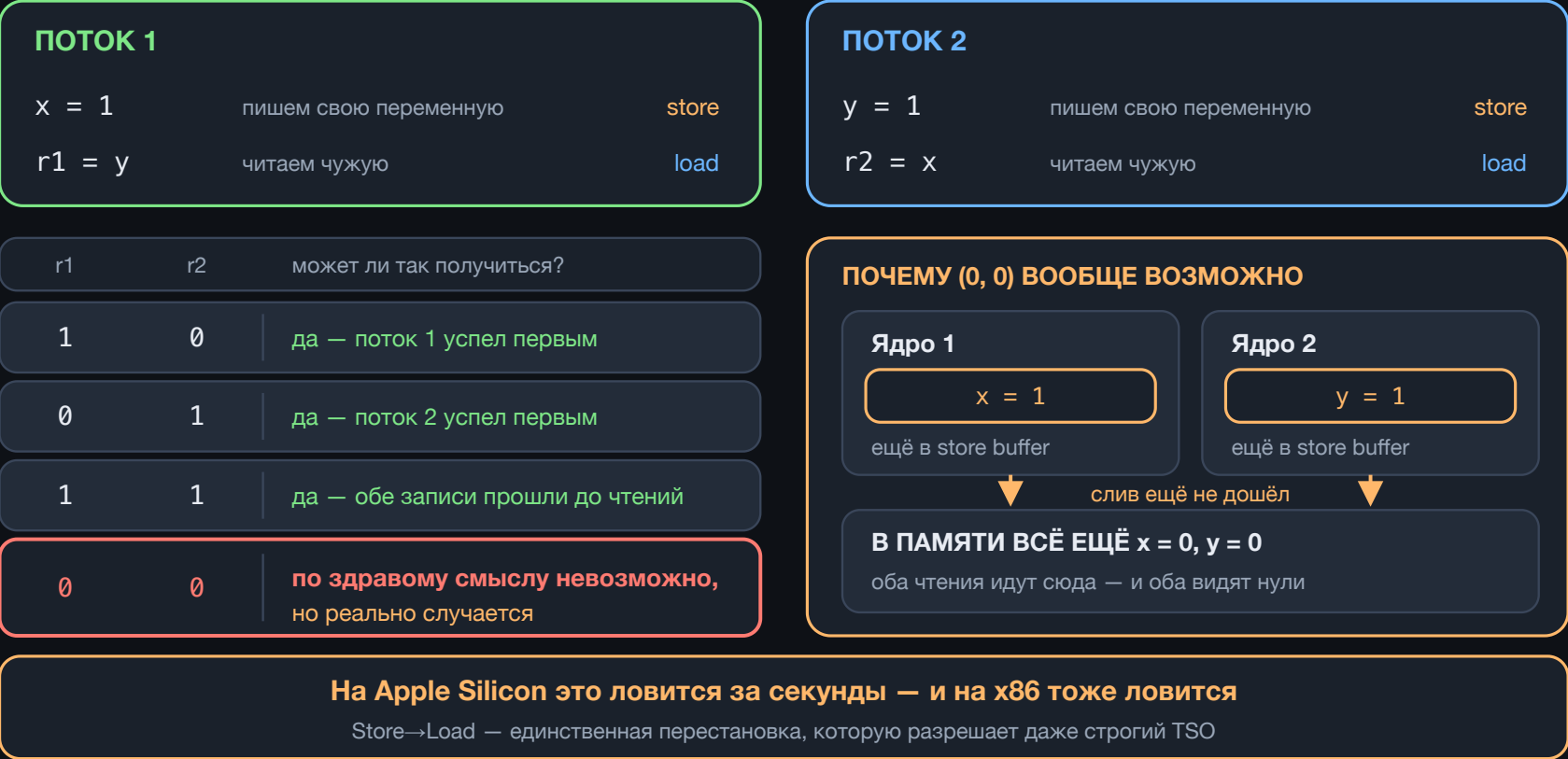


Запись ждёт в буфере, чтение идёт мимо — снаружи это выглядит как перестановка

Результат, которого «не может быть»

ТЕСТ STORE BUFFER, ОН ЖЕ ПРИМЕР ДЕККЕРА

изначально $x = 0$ и $y = 0$; $r1$ и $r2$ — локальные переменные, каждая в своём потоке



Что железо разрешает переставлять

ЧТО ЖЕЛЕЗО РАЗРЕШАЕТ ПЕРЕСТАВЛЯТЬ

две соседние операции с памятью: «можно» = процессор вправе выполнить их в обратном порядке

что за чем идёт в коде	Load → Load	Load → Store	Store → Store	Store → Load
x86-64 TSO — сильная модель Intel, AMD	нельзя порядок сохранён	нельзя порядок сохранён	нельзя порядок сохранён	МОЖНО виноват store buffer
ARM / Apple Silicon слабая модель POWER, RISC-V — такие же	МОЖНО без барьера — да	МОЖНО без барьера — да	МОЖНО без барьера — да	МОЖНО без барьера — да

Код, который годами работал на x86, начинает падать на ARM — и это не баг компилятора

А ЕЩЁ ЕСТЬ КОМПИЛЯТОР

Он переставляет обращения к памяти сам, независимо от железа — даже на x86.

ЧТО УСМИРЯЕТ ОБОИХ

Атомики и барьеры памяти: они запрещают перестановку и железу, и компилятору.

x86-TSO против ARM: одна разрешённая перестановка против четырёх

Sequential consistency: идеал Лэмпорта

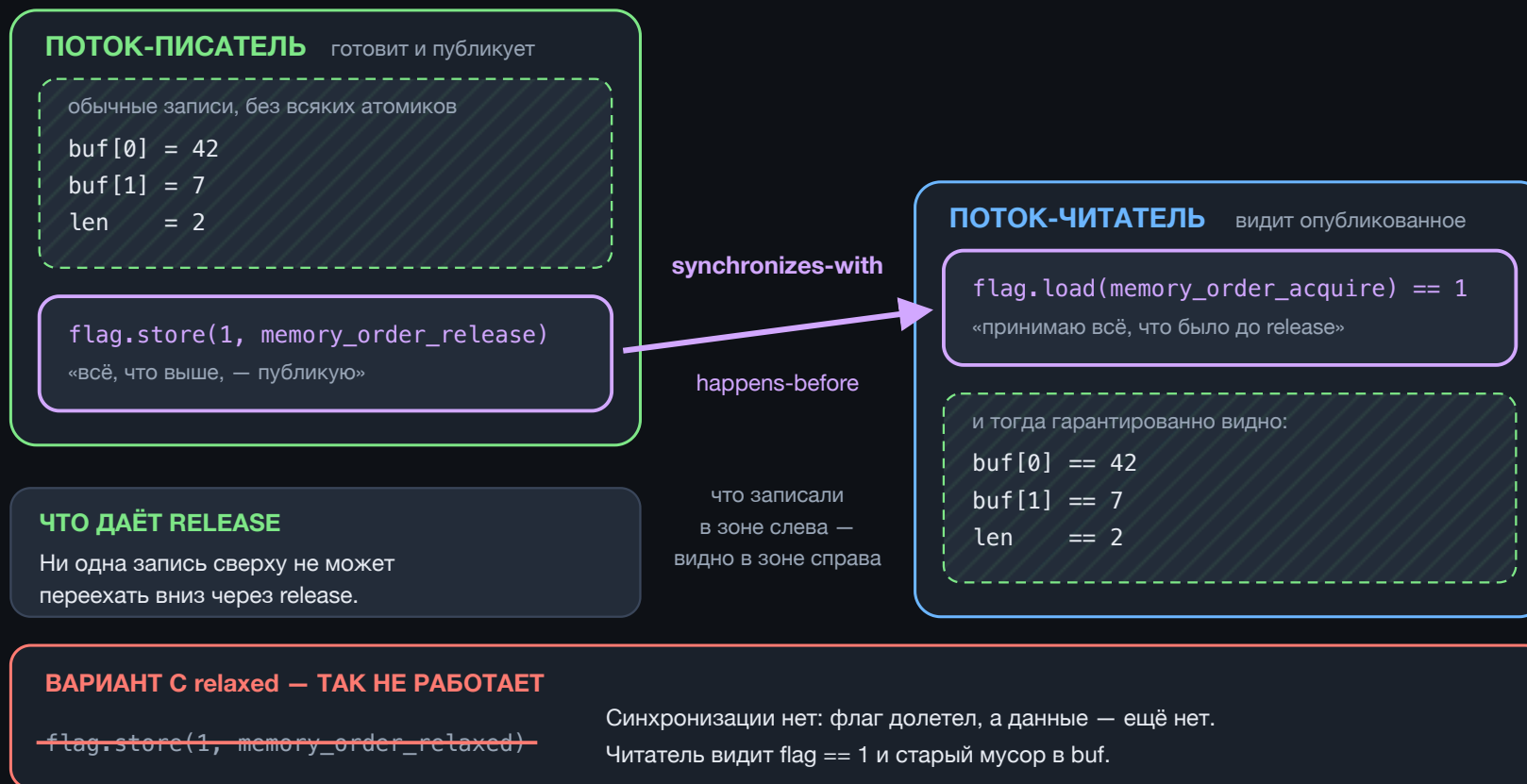
- Лэмпорт, 1979: результат такой, **как если бы** все операции всех потоков шли одной общей лентой.
- Внутри ленты операции каждого потока стоят в программном порядке.
- Ровно та модель, которую вы интуитивно держите в голове.
- Железо её даром не даёт: пришлось бы на каждой записи ждать когерентности.
- Поэтому: слабая модель плюс барьеры. SC продаётся поштучно.

happens-before: единственная валюта гарантий

- **sequenced-before** — порядок внутри одного потока, по тексту программы.
- **synchronizes-with** — release-запись и acquire-чтение, которое прочитало **именно это значение**.
- **happens-before** — транзитивное замыкание двух предыдущих отношений.
- Нет стрелки happens-before — нет никаких гарантий видимости.
- Это не время на часах, а стрелка в графе.

release / acquire: как публиковать данные

RELEASE / ACQUIRE: КАК ПРАВИЛЬНО ОПУБЛИКОВАТЬ ДАННЫЕ



Release публикует всё, что записано выше; acquire принимает всё, что было до него

Гонка данных — это неопределённое поведение

Гонка данных (data race) — четыре условия одновременно:

1. два обращения к одной ячейке памяти,
 2. хотя бы одно из них — запись,
 3. не оба обращения атомарные,
 4. между ними нет happens-before.
- В C++ такая программа — **UB**: компилятор вправе сделать что угодно.
 - В Go гонка не UB, но результат не определён, а широкие значения (интерфейс, срез) рвутся на куски.

`std::memory_order`: словарь из шести слов

- `relaxed` — только атомарность. Ни барьеров, ни обещаний про соседей.
- `acquire` (на чтении) — написанное **после** не уедет вверх через него.
- `release` (на записи) — написанное **до** не уедет вниз через него.
- `acq_rel` — для read-modify-write: CAS, `fetch_add`.
- `seq_cst` — всё перечисленное плюс единый глобальный порядок. Дефолт.
- `consume` — де-факто мёртв, компиляторы усиливают его до `acquire`.

Публикация данных: одна идея на трёх языках

```
// C++: порядок выбираем руками  
std::atomic<Cfg*> g{nullptr};  
g.store(p, std::memory_order_release);           // писатель  
Cfg* p = g.load(std::memory_order_acquire);      // читатель
```

CPP

```
// Go: выбора нет, atomic всегда самый строгий  
var g atomic.Pointer[Cfg]  
g.Store(p)  
p := g.Load()
```

GO

```
// Java: volatile – это и есть release/acquire  
volatile Cfg g;  
g = p;                               // release  
Cfg p = g;                           // acquire
```

JAVA

- Схема везде одна: заполнили объект, опубликовали указатель одной записью.

Когда `relaxed` действительно безопасен

```
std::atomic<uint64_t> hits{class="hl-n">0};  
  
void on_request() {                                     // горячий путь  
    hits.fetch_add(class="hl-n">1, std::memory_order_relaxed);  
}  
uint64_t total() {                                     // читаем, когда все потоки  
    return hits.load(std::memory_order_relaxed); // уже завершились  
}
```

CPP

- Счётчик **ничего не публикует**: по нему никто не решает, что читать.
- Правило-фильтр: решают по значению — нельзя; значение важно само по себе — можно.

Во что компилируется порядок

Что просим	x86-64	ARM64
обычные чтение и запись	mov	ldr / str
acquire -чтение	mov — порядок даром	ldar
release -запись	mov — порядок даром	stlr
seq_cst -запись	xchg (неявный lock)	stlr
отдельный барьер fence(seq_cst)	mfence	dmb ish

- На x86 acquire и release **бесплатны**: порядок уже обеспечен TSO.
- На ARM64 порядок — это суффикс инструкции, а не отдельный барьер.
- Замер на M4: fetch_add relaxed — 1.79 нс, seq_cst — 1.78 нс. Поровну.

Double-checked locking: сломанная классика

```
static Singleton* inst = nullptr;           // обычный указатель!

Singleton* get() {
    if (!inst) {                             // (1) чтение без синхронизации
        std::lock_guard lk(m);
        if (!inst) inst = new Singleton();  // (2) публикация без release
    }
    return inst;
}
```

CPP

- (1) и (2) — гонка данных, а значит UB.
- Поток увидит ненулевой `inst` раньше, чем конструктор дописал поля.

Ленивая инициализация: берите готовое

```
Singleton& get() { static Singleton s; return s; } // C++11: потокобезопасно
```

CPP

```
var once sync.Once
once.Do(func() { inst = newSingleton() })
```

GO

```
static CFG: OnceLock<Cfg> = OnceLock::new();
let c = CFG.get_or_init(|| Cfg::load());
```

RUST

- В Java — `enum`-синглтон или `holder`-идиома; в Python модуль грузится один раз.
- Увидели у себя «двойную проверку» — значит, есть готовый примитив.

Модель памяти Go: сознательно скучная

- `go f()` — всё, что было до запуска, видно новой горутине.
- Отправка в канал происходит раньше завершения соответствующего приёма.
- Заккрытие канала — раньше приёма нулевого значения из него.
- `Unlock` — раньше последующего `Lock`; `once.Do` — раньше для всех.
- Все операции `sync/atomic` — последовательно согласованные.

«Если вам приходится читать этот документ, чтобы понять поведение своей программы, вы слишком умничаете. Не умничайте.» — go.dev/ref/mem

Итог части 6

- Ваш код переписывают трое: компилятор, процессор, подсистема памяти.
- Единственная валюта гарантий — **happens-before**. Нет стрелки — нет гарантии.
- Сильный x86-TSO прощает ошибки, слабый ARM64 — нет.
- По умолчанию: мьютексы, каналы и `seq_cst`.
- `relaxed` — только с замером и комментарием, почему это безопасно.
- Самодельную синхронизацию всегда гоняем под санитайзером.

7. Кеши процессора: где живёт производительность

Здесь заканчивается теория и начинается железо.

- почему процессор половину жизни **ждёт память**
- физика: отчего DRAM медленная, а кеш быстрый
- кеш-линия, локальность, префетчер и четыре живых демки
- когерентность кешей (cache coherence) и цена общей записи
- как потерять всю выгоду восьми потоков на одной переменной

Стена памяти: две кривые, которые разошлись

СТЕНА ПАМЯТИ (memory wall)

процессор ускорялся десятилетиями, память — почти нет



Пропускная способность (ГБ/с) выросла отлично.

А ЗАДЕРЖКА одного обращения — почти нет.

Промех в память = 200–300 тактов простоя.

Кеш придуман, чтобы прятать именно задержку.

Процессор ускорялся десятилетиями, задержка памяти — почти нет

- Один поход в память — **сотни тактов** простоя ядра

- Полоса растёт, задержка стоит: DDR5 CL30 и DDR4 CL16 — одинаковые 10 нс

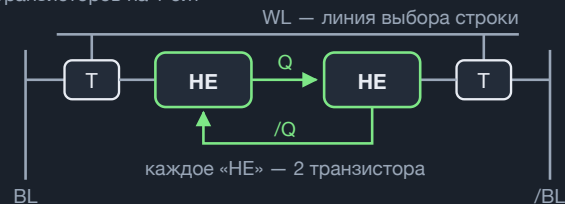
Почему память медленная: физика одной ячейки

ПОЧЕМУ КЕШ БЫСТРЫЙ, А ПАМЯТЬ МЕДЛЕННАЯ

всё решает устройство одной ячейки

SRAM — из неё сделан кеш

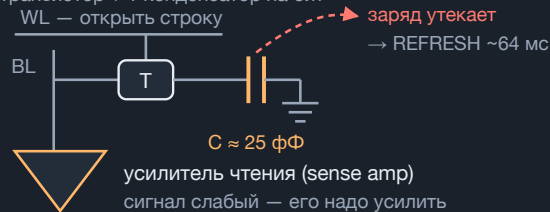
6 транзисторов на 1 бит



- два инвертора в кольце держат бит сами — пока есть питание, REFRESH не нужен
- чтение HE разрушает бит
прочитал — и ничего не надо восстанавливать
- 6 транзисторов на бит: площадь примерно в 6 раз больше, чем у ячейки DRAM
- поэтому SRAM мало — килобайты и мегабайты

DRAM — оперативная память

1 транзистор + 1 конденсатор на бит



- заряд утекает сам собой:
REFRESH всего массива каждые ~64 мс
- чтение РАЗРУШАЕТ бит — заряд стекает на разрядную линию (BL) — усилитель вернёт его
- сначала открыть строку (row activation, tRCD), и только потом выбрать колонку (CAS)
- зато дешево: 1 бит = 1 транзистор → гигабайты

Сотня наносекунд у DRAM — это не лень инженеров, а физика:

крошечный заряд на конденсаторе, длинная разрядная линия, слабый сигнал и обратная запись после каждого чтения.

Ячейка DRAM — это конденсатор, ячейка SRAM — шесть транзисторов

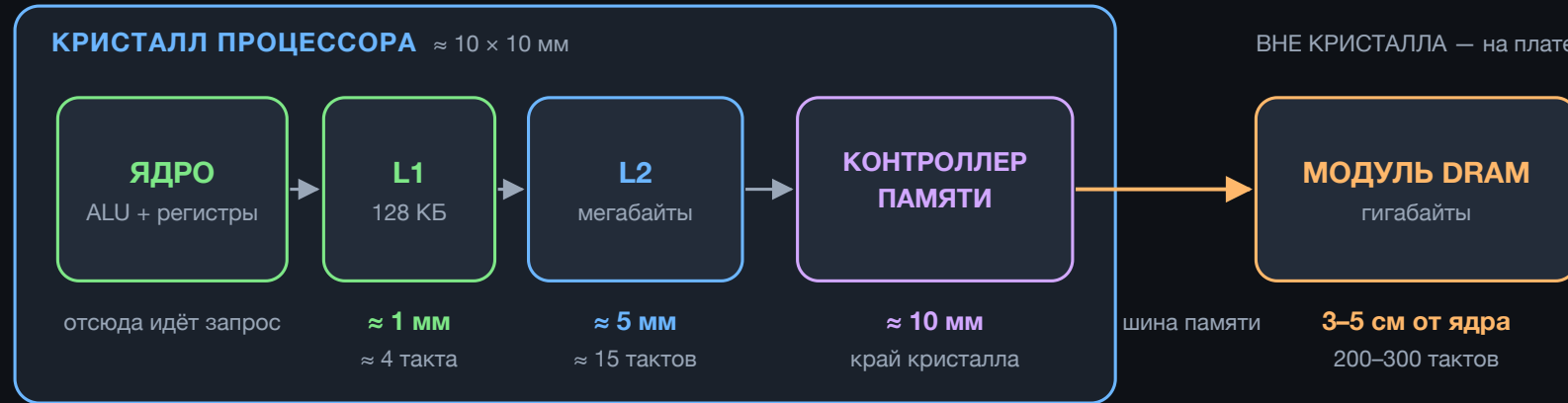
- Сам массив DRAM отдаёт данные примерно за **10 нс**

- Остальные 60–90 нс — дорога: контроллер, шина, когерентность, трансляция адреса

Почему кеш быстрый: расстояние не обманешь

ПОЧЕМУ КЕШ ОБЯЗАН БЫТЬ РЯДОМ С ЯДРОМ

расстояние нельзя обмануть



1 такт на 4 ГГц = 0.25 нс

за это время свет проходит 7.5 см,
а сигнал по линии на кристалле — 2–4 см
и это ещё без RC-задержек самих проводников

Расстояние не обманешь.

Можно только не ходить далеко —
кеш быстрый не «по волшебству»,
он просто лежит вплотную к ядру.

Почему L1 маленький: большой массив дольше адресовать, и сигнал внутри него бежит дольше.

Пока сигнал идёт до памяти, ядро успело бы сделать сотни операций

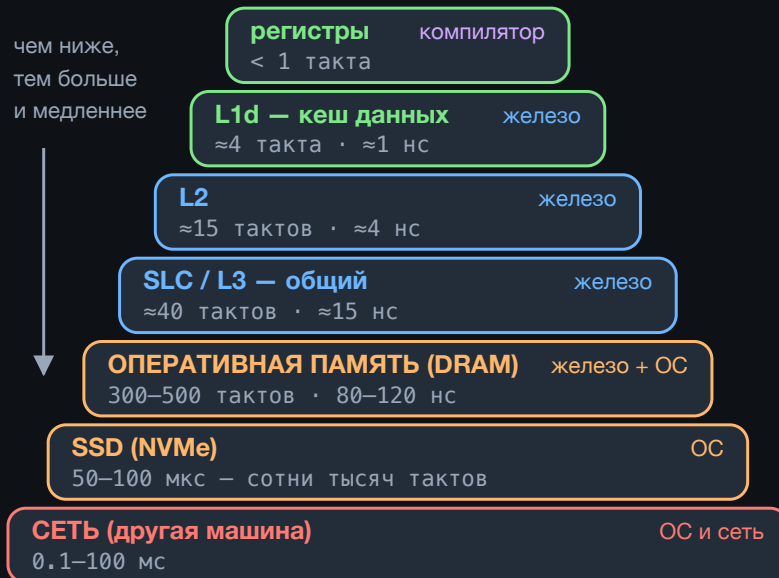
- L1d: 128 КБ у M4, 48 КБ у AMD Zen 5. Больше **никто не делает**

- Ограничение не цена, а время: большой массив дольше адресовать

Пирамида памяти: цена одного обращения

ПИРАМИДА ПАМЯТИ: цена одного обращения цифры для Apple M4 и типичного x86

УРОВЕНЬ · ЗАДЕРЖКА · КТО УПРАВЛЯЕТ



ОБЪЁМ

сотни байт

128 КБ на Р-ядро M4
32–48 КБ у x86

1–16 МБ
у M4 — на кластер ядер

десятки МБ
один на все ядра

гигабайты

терабайты

весь остальной мир

ЕСЛИ БЫ ОДНА
НАНОСЕКUNDA
БЫЛА СЕКУНДОЙ

доли секунды

1 секунда

4 секунды

15 секунд

2 минуты

сутки-двое

3–4 месяца

Каждая ступень вниз — в 3–10 раз дороже предыдущей.

Кеш спасает только потому, что программы ходят в память не случайно: рядом лежащее нужно почти всегда.

Каждая ступень вниз — в 3–10 раз дороже предыдущей

- Свои числа смотрите сами: macOS — `sysctl -a | grep cachesize`, Linux — `lscpu`

Четыре такта до L1, четыреста до памяти

уровень	Apple M4	AMD Zen 4	Intel Lion Cove
попадание в L1d	4 такта	4 такта	4 такта
L2	~22 такта	14 тактов	17 тактов
последний общий уровень	десятки-сотни	~50 тактов	~51 такт
DRAM	~440 тактов	~400 тактов	~400 тактов

- Три разные архитектуры — **одни и те же порядки величин**
- Потому что упирается не в схемотехнику, а в физику DRAM

Шпаргалка из интернета устарела

величина	шпаргалка	реальность 2026
чтение из L1	4.5 нс	0.8 нс
чтение из L2	19.5 нс	5 нс
мьютекс lock + unlock	75 нс	4.3 нс
чтение из памяти	100 нс	70–100 нс
пакет вокруг Земли	150 мс	150 мс

- Устарело всё, что делают **транзисторы**; физика не сдвинулась
- Значит, разрыв между ядром и памятью не сократился, а **вырос**

Кеш-линия: память возит коробками

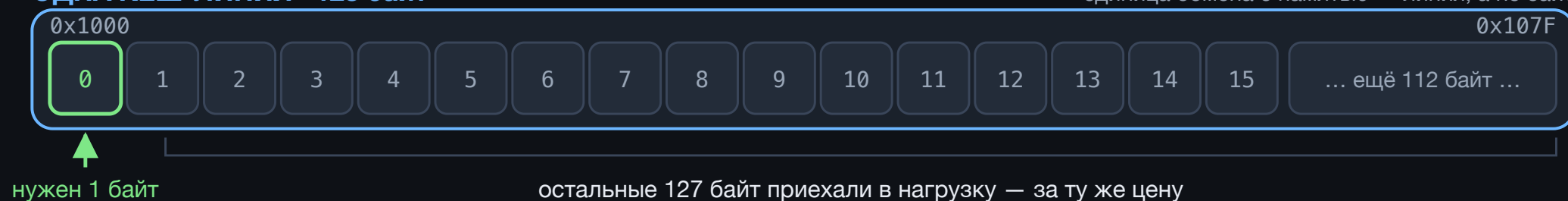
КЕШ-ЛИНИЯ: ПАМЯТЬ ВОЗИТ КОРОБКАМИ, А НЕ БАЙТАМИ

Программа просит ОДИН байт: `char c = *(char*)0x1000;`

— а из памяти приезжает целая линия

ОДНА КЕШ-ЛИНИЯ · 128 байт

единица обмена с памятью — линия, а не байт



x86-64 — линия 64 байта

16 × int32 или 8 × double за один поход
про эту цифру написаны все книги и статьи

Apple Silicon (M4) — 128 байт

`sysctl hw.cachelinesize → 128`
вдвое шире: чужие примеры с 64 байтами ломаются

ПОСЛЕДОВАТЕЛЬНЫЙ ОБХОД — почти даром

один промах на 32 int32 — остальное уже приехало
префетчер видит шаг и подвозит следующие линии

СЛУЧАЙНЫЙ ОБХОД — платим за каждую линию

1 нужный байт на 128 привезённых
в 32 раза больше трафика на тех же данных

Просишь один байт — приезжает целая линия

- Линия неделима: её целиком везут, целиком хранят и целиком отбирают у соседа

- Линейный проход по `int32`: один промах на 32 элемента, остальное даром

Как кеш находит линию

КАК КЕШ НАХОДИТ ДАННЫЕ: ТЕГ, ИНДЕКС, СМЕЩЕНИЕ

пример: L1d 32 КБ · 8 путей · линия 64 байта → 64 множества · адрес 48 бит · цвет показывает, за что отвечает часть адреса



Адрес делится на тег, индекс и смещение

- Вытеснение псевдо-LRU: адреса с шагом ровно 4096 бьются в одно множество

- Запись — **write-back** плюс грязный бит (dirty): в память сбрасываем при вытеснении

Локальность и префетчер

- **Временная (temporal)**: к тому же адресу обратятся снова
- **Пространственная (spatial)**: следом прочитают соседний адрес
- Префетчер (hardware prefetcher) ловит постоянный шаг и возит линии заранее
- Любит: линейный обход, фиксированный шаг, несколько потоков чтения
- Бессилен: случайные адреса, прыжки по указателям, шаг больше страницы
- Предсказуемый доступ дешевле «умного», но хаотичного

ДЕМО 1: обход матрицы 4096×4096

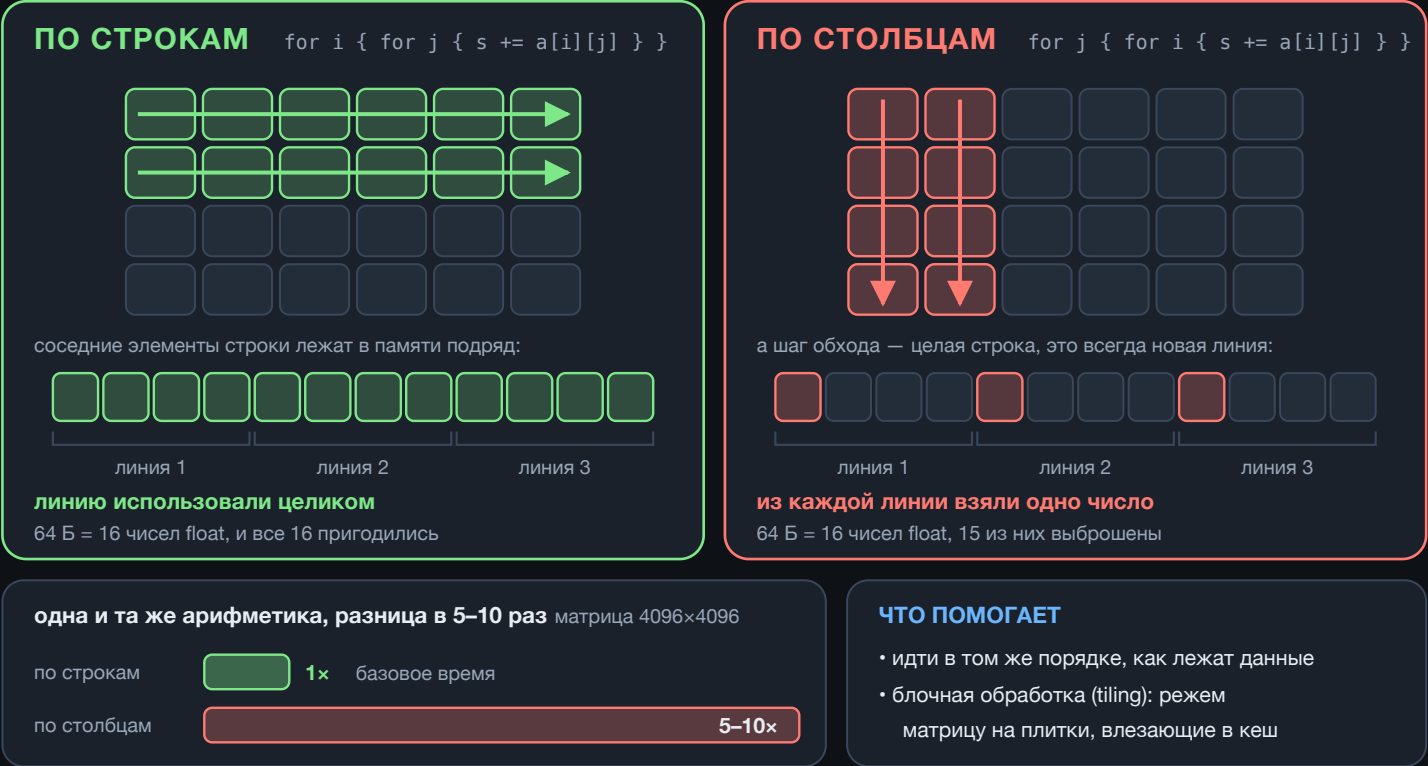
```
float s = class="hl-n">0; // ПО СТРОКАМ: шаг 4 байта
for (int i = class="hl-n">0; i < N; ++i)
    for (int j = class="hl-n">0; j < N; ++j) s += m[i*N + j];

float s2 = class="hl-n">0; // ПО СТОЛБЦАМ: шаг 16 КБ
for (int j = class="hl-n">0; j < N; ++j)
    for (int i = class="hl-n">0; i < N; ++i) s2 += m[i*N + j];
```

CPP

- Те же 16.7 млн чтений и сложений. Отличается только **порядок**

ДЕМО 1: те же операции, в шесть раз дольше

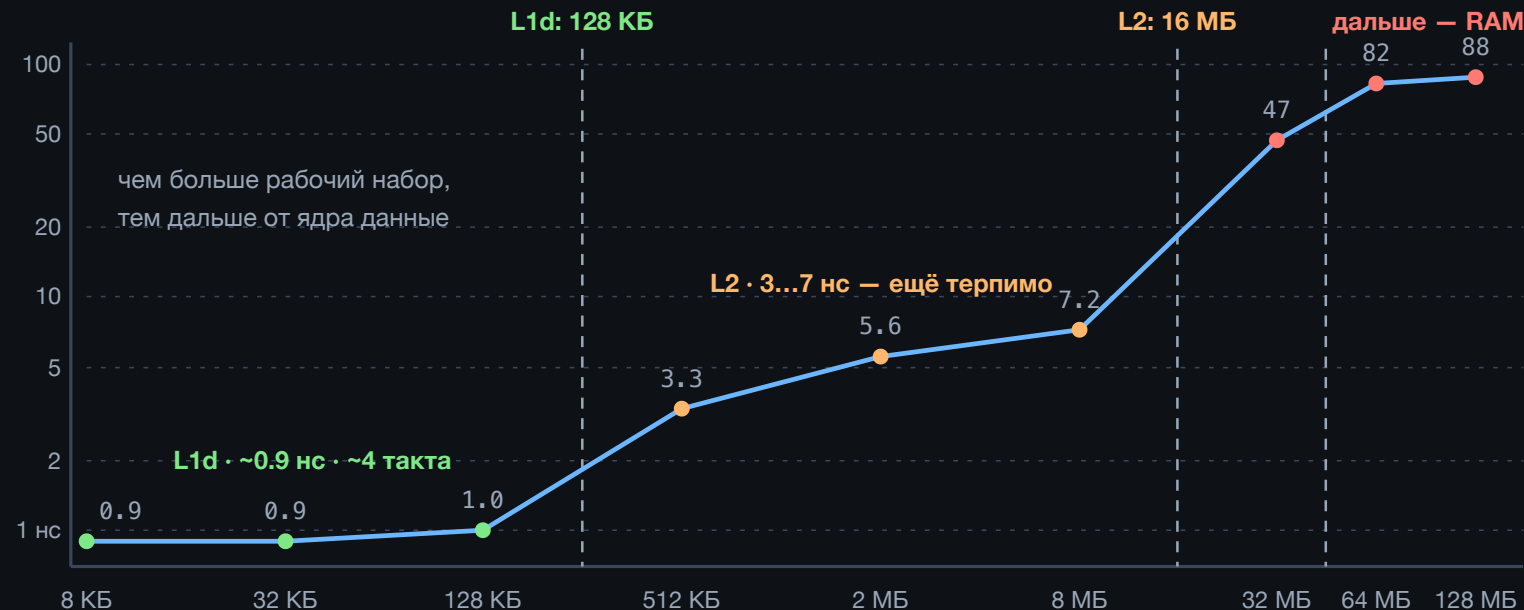


- По столбцам прокачали 2 ГБ линий ради 64 МБ данных

ДЕМО 2: лесенка латентности

ЛЕСЕНКА ЛАТЕНТНОСТИ: КАРТА ПАМЯТИ, СНЯТАЯ ЗАМЕРОМ

по вертикали — наносекунды на одно обращение (лог. шкала), по горизонтали — размер рабочего набора (лог. шкала)



Замер на Apple M4: случайный обход цикла указателей (pointer chasing) — префетчер бессилен, видно чистую латентность

Ступенька на 32 МБ — набор уже не влезает в L2: часть обращений уходит в общий кеш, часть прямо в память

Этот график каждый может построить сам за минуту — он и есть карта памяти вашей машины

Границы кешей видно глазами: 128 КБ, 16 МБ, дальше память

- Профиль вашего железа, снятый программой на тридцать строк

- При линейном обходе ступенек почти нет: префетчер их прячет

Список против вектора: **pointer chasing**

- 4 млн элементов: `std::vector<int>` — **0.21 мс**, `std::list<int>` — **3.18 мс**
- Узел списка: 8 + 8 байт указателей на 4 байта полезных данных
- Адрес следующего узла известен только после загрузки текущего
- Зависимые загрузки: ни префетчер, ни внеочередное исполнение не спасают
- **$O(1)$ в учебнике $\neq$ «быстро на железе»**: константа живёт в кешах

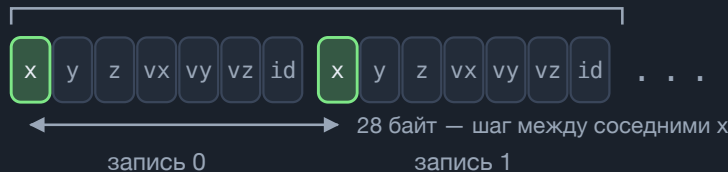
AoS против SoA: как разложить частицы

AoS — массив структур

```
struct P { float x,y,z, vx,vy,vz; int id; };
```

задача: просуммировать x y всех частиц

одна линия кеша, 64 Б — это примерно 2 записи



в линию влезло всего 2 нужных числа,
остальное — vx, vy, vz, id: балласт

полезных байт в линии: ~14%

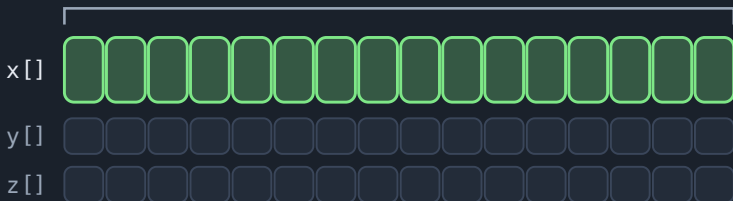


SoA — структура массивов

```
float x[N], y[N], z[N], vx[N], vy[N], vz[N];
```

та же задача: просуммировать x

одна линия кеша, 64 Б — это 16 значений x подряд



в линию влезли 16 нужных чисел подряд,
а плотный массив ещё и векторизуется (SIMD)

полезных байт в линии: 100%



Раскладка данных в памяти — это тоже алгоритм

тот же цикл, та же математика — а времени в разы больше

Одна линия: два нужных числа против шестнадцати

- 8 млн частиц, сдвигаем координаты: AoS **6.1 мс**, SoA **3.4 мс**

- SoA — готовый вход для SIMD: соседние значения ложатся в вектор как есть
- Обратная сторона: читать код труднее, «частицу целиком» доставать дороже

Тайлинг: режем задачу под размер кеша

```
// c = a * b, N = 1024, блок B = 64 -> 64*64*4 = 16 КБ на блок
for (int ii = 0; ii < N; ii += B)
    for (int kk = 0; kk < N; kk += B)
        for (int jj = 0; jj < N; jj += B)
            for (int i = ii; i < ii + B; ++i)
                for (int k = kk; k < kk + B; ++k) {
                    float aik = a[i*N + k];
                    for (int j = jj; j < jj + B; ++j) c[i*N + j] += aik * b[k*N + j];
                }
```

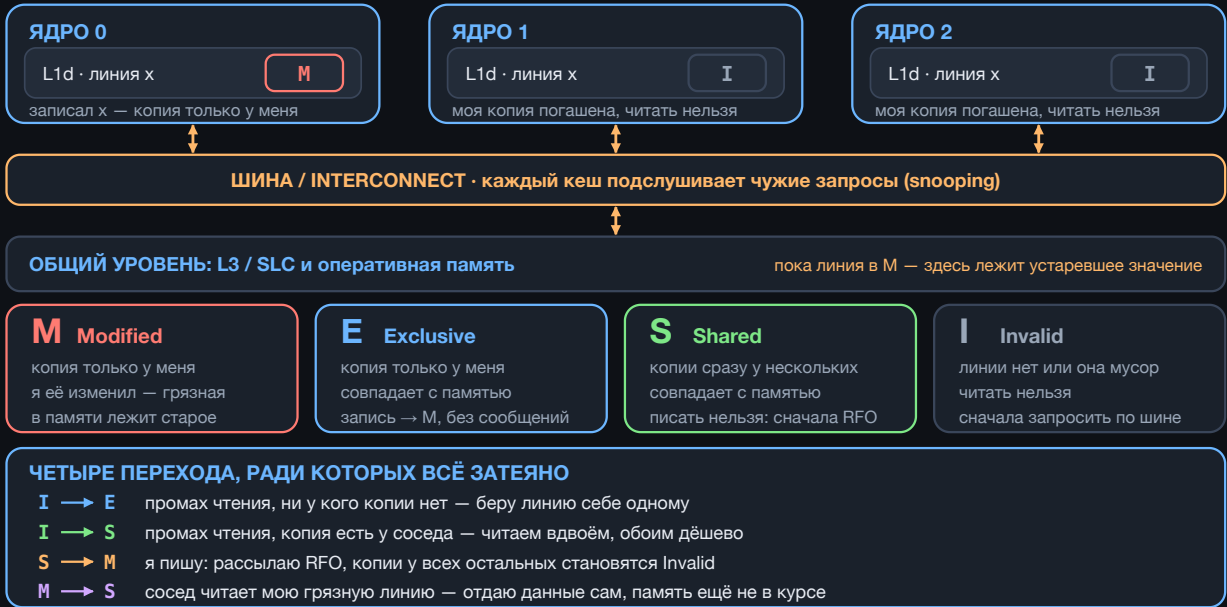
CPP

- Наивный `ijk` — **818 мс**, блочный — **49 мс**. Сложность та же, $O(N^3)$

Ядер много, кешей тоже: когерентность

КОГЕРЕНТНОСТЬ КЕША: ПРОТОКОЛ MESI

у каждого ядра своя копия линии — железо следит, чтобы никто не прочитал протухшее



Внутри кристалла работает настоящий распределённый протокол согласования — только на наносекундах

У каждого ядра своя копия линии, железо следит за согласованностью

- Ядро 0 записало **x** в свой L1. Откуда об этом узнает ядро 1?

- Ответ железа: протокол когерентности, кеши переговариваются сами
- Протухшую копию не прочитает никто: **корректность бесплатна, скорость нет**

RFO: чтобы писать, линию надо отобрать

RFO И ПИНГ-ПОНГ: РЕАЛЬНАЯ ЦЕНА ОБЩЕЙ ПЕРЕМЕННОЙ

одна кеш-линия с переменной x, два ядра, три кадра подряд



Три кадра: оба читают, один пишет, линия переезжает

- **RFO (Read For Ownership)**: перед записью линию забирают в монополию

- Чтение общей линии дёшево, запись дорога — вся проблема растёт отсюда
- Запись в свою, никем не разделяемую линию почти бесплатна

Сколько стоит переезд линии между ядрами

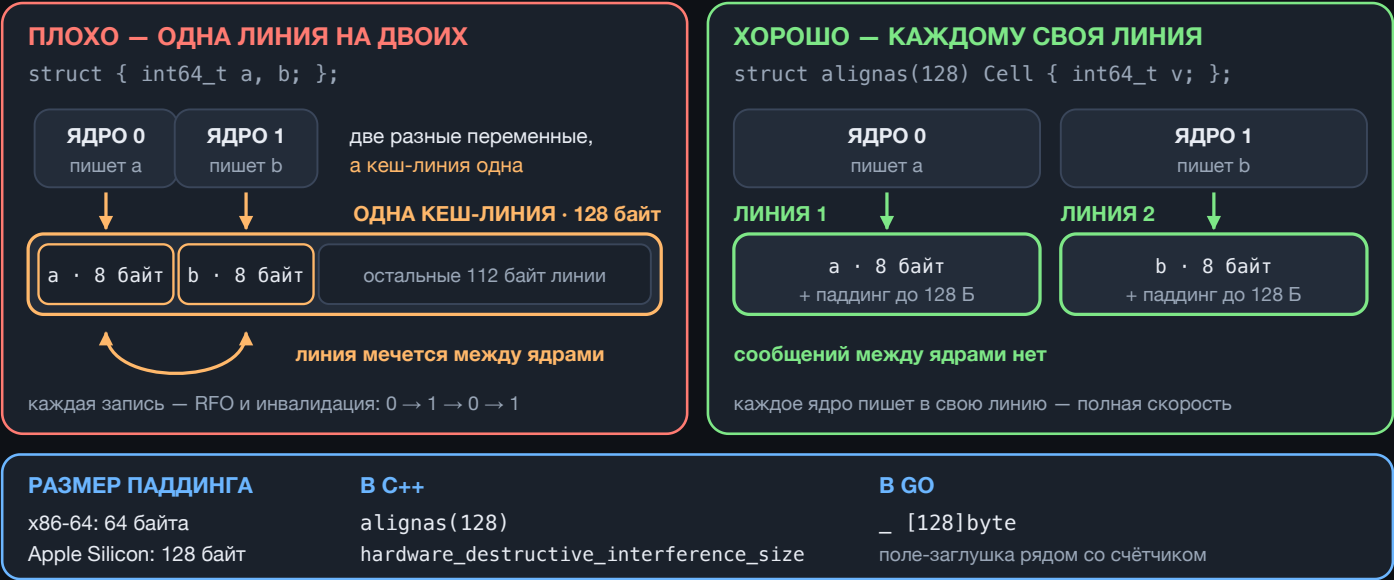
откуда куда	задержка
M4, соседние ядра (замер)	37 нс ≈ 164 такта
M1 Pro, производительные / энергоэффективные	40 / 53 нс
M1 Pro, между кластерами	145 нс
AMD Zen 3, внутри чиплета	17 нс
AMD Zen 3, между чиплетами	85 нс
AMD Zen 5 (9950X), между чиплетами	~200 нс

- Внутри кластера — десятки наносекунд, наружу — в разы дороже

Ложное разделение (false sharing)

ЛОЖНОЕ РАЗДЕЛЕНИЕ (FALSE SHARING)

две независимые переменные случайно оказались в одной кеш-линии — и ядра начинают мешать друг другу



- Программа корректна: гонок нет, санитайзер молчит, тесты зелёные
- Симптом: добавили потоков — стало не быстрее, а медленнее

ДЕМО 3: два счётчика — рядом и врозь

```
struct Bad { std::atomic<int64_t> a{class="hl-n">0}, b{class="hl-n">0}; }; // одна л
struct alignas(class="hl-n">128) Cell { std::atomic<int64_t> v{class="hl-n">0}; }; // каждом
```

```
type Counter struct {
    v int64
    _ [120]byte // добиваем структуру до 128 байт
}
```

- Два потока по 50 млн инкрементов: **341 → 80 мс** (C++), **310 → 81 мс** (Go)

Сколько байт паддинга: 64, 128 или 256?

источник (одна и та же машина)	ответ
<code>sysctl hw.cachelinesize</code>	128
Go, <code>CacheLinePadSize</code> для arm64	128
C++ <code>hardware_destructive_interference_size</code>	256
прямой замер гранулярности	64

- Обрыв времени случается между 32 и 64 байтами, а не между 64 и 128
- Практика: **128 байт безопасны везде**, на x86-64 линия всегда 64

Где водится false sharing и как лечить

- Массив по номеру воркера: `res[worker_id]++` — классика и главный источник
- Голова и хвост очереди в одной структуре: их пишут разные потоки
- Флаг `stop` рядом с горячим счётчиком в общей структуре
- Лечение первым делом: локальный счётчик и **одна** запись в конце
- Если не выходит — паддинг или отдельная структура на поток
- Ловить: `perf c2c` на Linux, на macOS — честный замер с паддингом и без

Сами примитивы дешёвые

операция без конкуренции (замер на M4)	нс	тактов
обычный <code>++</code> в L1	0.33	1.4
<code>atomic fetch_add</code> , relaxed	1.79	7.9
<code>atomic fetch_add</code> , seq_cst	1.78	7.8
цикл CAS	1.82	8.0
мьютекс lock + unlock	4.25	18.7

- Атомик дороже обычной записи всего в пять раз
- `relaxed` и `seq_cst` стоят одинаково: модель памяти — про корректность

ДЕМО 4: добавили потоков — стало медленнее

потоков	одна общая линия	каждому своя	замедление
2	3.60 нс	0.92 нс	3.9x
4	10.92 нс	0.49 нс	22x
8	16.64 нс	0.46 нс	36x

- Суммарно слева: 278 → 92 → 60 млн оп/с. **Отрицательная масштабируемость**
- Паддинг не поможет: переменная общая по делу, это true sharing

Шардирование: своя линия каждому

```
type shard struct {  
    mu sync.Mutex  
    m  map[string]int  
    _  [96]byte // разводим шарды по разным кеш-линиям  
}  
type Sharded [256]shard  
  
func (s *Sharded) at(k string) *shard { return &s[hash(k)%256] }
```

GO

- Каждый пишет в своё, общая картина собирается в конце
- Так устроены конкурентные словари, sharded-кеши, счётчики на каждое ядро в ОС

Как всё это мерить

- Linux: `perf stat -e cycles,instructions,cache-misses,LLC-load-misses ./app`
- macOS: Instruments, `xctrace record --template 'CPU Counters' --launch ./app`
- Go: `go test -bench . -count=10` и `benchstat`
- Главная метрика — **IPC** (инструкций за такт): ниже единицы — ядро ждёт память
- Компилятор выбрасывает мёртвый код: `DoNotOptimize`, `KeepAlive`, запись наружу
- Методика: прогрев, 5–10 прогонов, медиана, ноутбук в розетке

Пять правил дружбы с кешем

1. **Ходите по памяти линейно** — порядок важнее числа операций
2. **Считайте линии, а не объекты** — 64 байта на x86, 128 на Apple Silicon
3. **Пишите в свою линию** — общая запись отбирает её у всех соседей
4. **Делите работу, а не переменные** — шарды и локальные аккумуляторы
5. **Не гадайте, мерьте** — интуиция про производительность врёт почти всегда

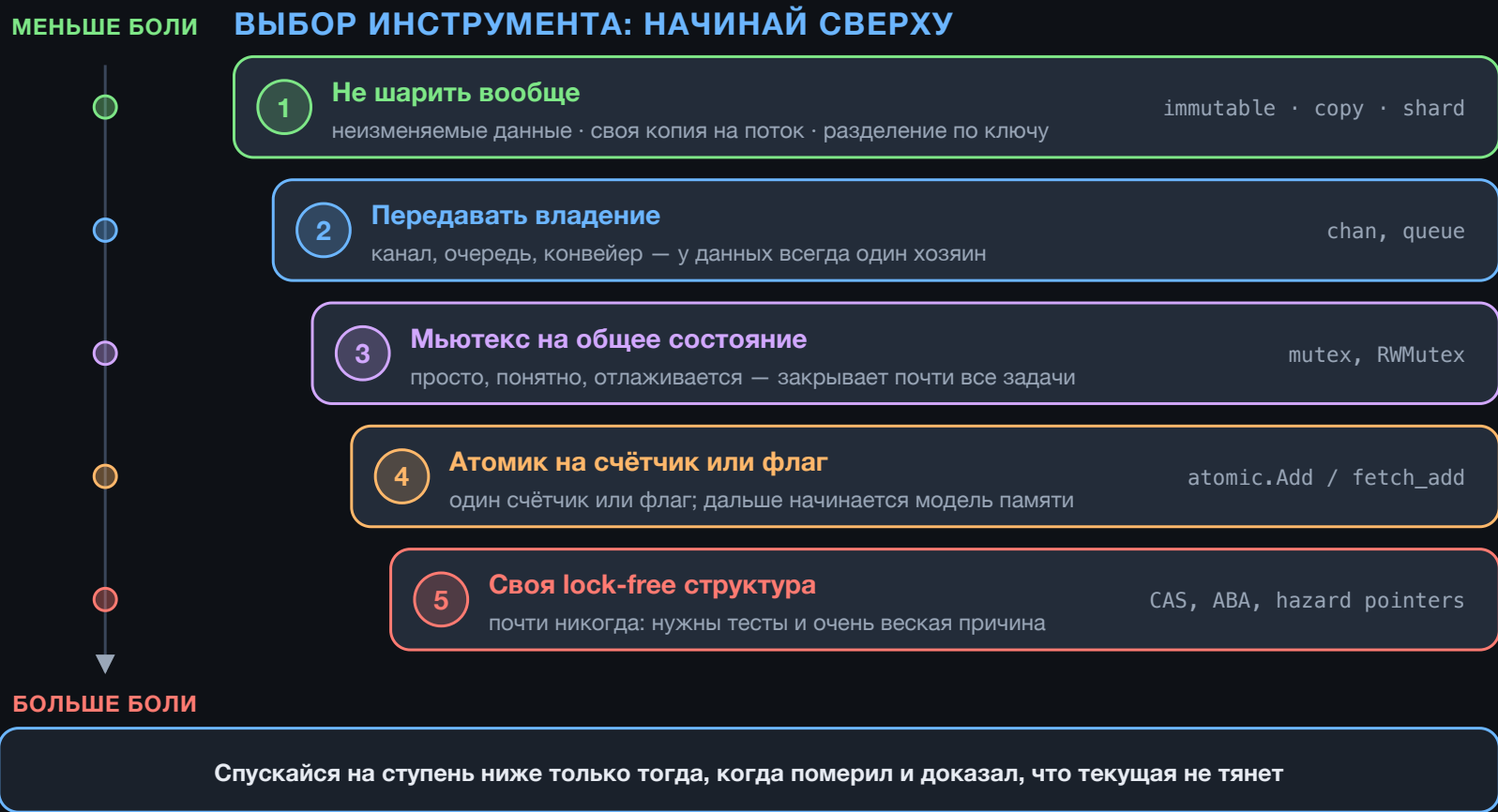
Многопоточность не спасёт программу, которая упёрлась в память. Сначала раскладка данных, потом потоки.

8. Практика, инструменты и здоровый смысл

«Правило оптимизации: не делай. Правило для экспертов: пока не делай.» — Майкл Джексон, «Principles of Program Design», 1975

- Семь частей были про то, **как** конкурентность устроена внутри
- Эта часть — про то, **что с этим делать в понедельник на работе**
- Порядок работ, чеклист ревью, типовые грабли, диагностика в бою

Лестница выбора инструмента



Пять ступеней: чем ниже спускаешься, тем больше боли и тем нужнее замер

Сначала корректно, потом быстро

- Порядок ровно один: **корректно → тест → профиль → оптимизация → снова тест**
- Профилируй, а не угадывай: интуиция про горячее место ошибается
- Параллелится 80% работы — потолок ускорения 5х, хоть на ста ядрах
- Сначала смотри на те 20%, которые не параллелятся

Оптимизация без замера — это не инженерия, это вера.

Замер: как не соврать себе

```
go test -bench=. -count=10 -benchtime=2s ./... > new.txt  
benchstat old.txt new.txt      # есть разница или это шум?
```

BASH

- Меряем **до и после**, на реальных данных, не меньше 10 повторов
- Берём медиану, а не среднее: выбросы портят среднее
- Разброс между прогонами больше эффекта — значит, эффекта нет
- На ноутбуке: воткнуть зарядку, закрыть браузер и звонилку
- Р- и Е-ядра дают разброс в разы на одном и том же бенчмарке

Где конкурентность не нужна

- **Порог запуска.** Горютина — сотни наносекунд, поток ОС — десятки микросекунд
- Единица работы должна быть заметно дольше собственного запуска
- **Уже упёрлись в диск или сеть?** Десять потоков будут ждать вдесятером
- **Батчинг часто бьёт потоки:** одна транзакция на 1000 строк вместо 1000 вставок
- Смена алгоритма или раскладки данных даёт больше, чем любые потоки

Шардирование: лучший способ не тормозить — не шарить

ПЛОХО

один счётчик на все потоки



true sharing — точка конкуренции

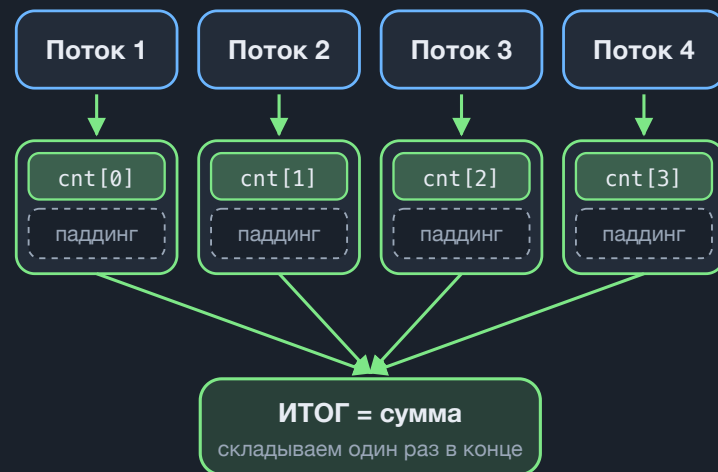
линия кеша мечется от ядра к ядру:
чтобы записать, ядро отбирает её у остальных

больше потоков → медленнее

четыре ядра гоняют одну линию по кругу

ХОРОШО

у каждого потока свой счётчик



нет общей линии — нет конкуренции

паddинг разводит счётчики по разным линиям (128 Б)

Тот же приём в больших структурах — sharded map

N независимых частей, у каждой свой мьютекс; часть выбираем по хешу ключа
так устроены почти все быстрые конкурентные словари

Общий счётчик против счётчика на поток: итог складываем один раз в конце

Чеклист ревью: у данных есть хозяин?

1. **Что именно защищает этот мьютекс?** Рядом с полем — `// guarded by mu`
2. **Есть ли зафиксированный порядок захвата**, если замков два?
3. **Кто закрывает канал?** Только отправитель, и ровно один
4. **Нет ли блокирующего вызова под мьютексом?** Сеть, диск, лог, чужой колбэк

Чеклист ревью: кто остановит и кто проверит?

1. Кто останавливает эту задачу и как она об этом узнает?
2. Передаётся ли `ctx` во все блокирующие вызовы и проверяется ли он?
3. Не копируется ли структура с мьютексом по значению? Это ловит `go vet`
4. Зелёный ли `-race` и есть ли стресс-тест на инвариант?

Габли Go: три классики

GO

```
// 1. Значимый получатель копирует мьютекс – ловит go vet  
func (c Counter) Inc() { c.mu.Lock(); c.n++; c.mu.Unlock() }
```

```
// 2. Add внутри горутины: Wait может успеть раньше Add  
for _, u := range users {  
    go func() { wg.Add(1); defer wg.Done(); send(u) }()  
}
```

```
// 3. Читателя у канала нет – горутина висит вечно  
go func() { ch <- work() }()
```


Грабли Go: что роняет процесс целиком

- `fatal error: concurrent map writes` — **не паника**, `recover` не ловит
- `append` в общий срез из горутин — гонка даже без роста массива
- `defer` в цикле копит вызовы до выхода из функции: замок не отпустится
- `case <-time.After(d)` в горячем `select` — новый таймер каждую итерацию

Грабли C++: время жизни опаснее гонок

```
void bad() {  
    int local = class="hl-n">42;  
    std::thread t([&] { use(local); }); // ссылка на кадр bad()  
    t.detach();                       // кадр умер, ссылка висит  
}  
  
void also_bad() {  
    std::thread t(work);  
    may_throw(); // исключение -> join не вызван -> std::terminate  
    t.join();  
}
```

CPP

- `[&]` плюс `detach` — висячая ссылка; по умолчанию бери `std::jthread`
- `shared_ptr`: счётчик ссылок атомарный, **сам объект — нет**

Одна ошибка — три языка

```
std::thread t([&] { use(local); });    // C++: компилятор молчит, это UB
t.detach();
```

CPP

```
go func() { use(local) }()           // Go: переменная уедет в кучу,
                                     // но горутину всё равно никто не ждёт
```

GO

```
thread::spawn(move || use(local));    // Rust: без move не соберётся вообще
```

RUST

Тестирование: race-детектор

- `-race` в CI обязателен: `go test -race ./... -count=5`
- В C++ то же самое: сборка с `-fsanitize=thread` отдельной джобой
- Санитайзер ловит только то, что **реально исполнилось** в этом прогоне
- Цена: время x5–15, память x5–10 — поэтому отдельным прогоном
- Зелёно без `-race` значит только, что гонки никто не искал

Стресс, фаззинг и флейки

- Стресс: N потоков по M операций, в конце — проверка инварианта
- Повторять и варьировать: `-count=100`, `-cpu=1,4,10`
- `-timeout` обязателен: дедлок должен падать дампом, а не висеть в CI
- Фаззинг (`go test -fuzz`): случайные последовательности операций
- `GOMAXPROCS=1` **не** делает Go детерминированным
- Флейк не перезапускаем, а расследуем

Высший пилотаж: DST и формальные модели

- **Детерминированное симуляционное тестирование** (deterministic simulation testing)
- Время, сеть, диск и планировщик подменяются симулятором
- Единственный источник случайности — seed: упал тест, воспроизводим байт в байт
- Так живут FoundationDB, TigerBeetle, Antithesis
- Младший брат в стандартной библиотеке Go — пакет `testing/synctest`
- **TLA+ и Spin:** перебор состояний протокола вместо примеров

Диагностика: pprof и профили ожидания

- `import _ "net/http/pprof"` даёт `/debug/pprof/` — только на приватный порт
- Профили `block` и `mutex` выключены по умолчанию, включаются явно
- `runtime.SetMutexProfileFraction(5)`, `runtime.SetBlockProfileRate(10000)`
- `go tool pprof -http=:8080 http://host:6060/debug/pprof/mutex`
- Тысяча горутин в `chan receive` — это утечка, а не нагрузка

Когда всё зависло намертво

- `SIGQUIT` (Ctrl-\ в терминале) — дампы всех горутин в stderr
- В контейнере то же самое: `kill -QUIT 1`; `GOTRACEBACK=all` расширяет дампы
- `curl 'host:6060/debug/pprof/goroutine?debug=2'` — стеки всех горутин
- `dlv attach PID`, дальше `goroutines -t`
- C++: `lldb -p PID` или `gdb -p PID`, затем `thread apply all bt`
- Метрики важнее профиля: `go_goroutines`, длина очереди, время ожидания замка

Кейс: сервис тормозит под нагрузкой

Симптом: $p99 = 180$ мс, RPS не растёт, а процессор загружен всего на 30%

1. **Метрики:** число горутин ползёт вверх, RPS плоский
2. **Дамп горутин:** ~800 штук стоят в `sync.(*Mutex).Lock`
3. **Mutex-профиль:** 92% ожидания на одном `RWMutex` кеша
4. **Чтение кода:** под замком — поход в сеть и разбор JSON
5. **Правки:** IO вынести из-под замка, кеш на 256 шардов, чтение под `RLock`
6. **Замер:** $p99\ 180 \rightarrow 12$ мс, борьба за замок (contention) ушла из топа

Итоги: как это устроено

1. Конкурентность — про структуру программы, параллелизм — про железо
2. Поток ОС дорог, горютина дешева, но не бесплатна
3. Гонка данных (data race) — это UB в C++ и рваные значения в Go
4. «Работает» не значит «корректно»: гонка умеет годами не проявляться
5. Мьютекс — не позор, а инструмент по умолчанию

Итоги: как не выстрелить себе в ногу

1. Не делите память — общайтесь; но канал внутри — тот же мьютекс
2. Атомик — про счётчик и флаг; lock-free сложен и не всегда быстрее
3. Порядок, который вы написали, — пожелание; гарантию даёт happens-before
4. Память не плоская: кеш-линия одна на соседние переменные
5. Сначала корректность, потом профиль, и только потом трюки

У любого разделяемого изменяемого состояния должен быть явный хозяин.

Что почитать и посмотреть

- **Anthony Williams**, «C++ Concurrency in Action», 2-е изд. — настольная для C++
- **Katherine Cox-Buday**, «Concurrency in Go» — паттерны и утечки
- **Herlihy, Shavit**, «The Art of Multiprocessor Programming» — теория и lock-free
- **Ulrich Drepper**, «What Every Programmer Should Know About Memory» — идеи живы
- **Доклады:** Pike «Concurrency Is Not Parallelism», Sutter «atomic Weapons»
- **Ссылки:** preshing.com · go.dev/ref/mem · [cppreference](https://cppreference.com) · godbolt.org

Семинар и домашка

- **Семинар:** собрать `materials/cpp/15_false_sharing.cpp` и `17_contention_sharding.cpp`
- Поймать гонку в `materials/go/02_race` под `-race` и починить тремя способами
- Снять mutex- и block-профиль с `materials/go/12_perf` через pprof
- **Домашка 1:** воркер-пул с остановкой по `ctx`, зелёные `go vet` и `-race -count=100`
- **Домашка 2:** кеш в трёх версиях (мьютекс, шарды, atomic с паддингом), график 1..10 потоков
- **Домашка 3:** отчёт на страницу — где нашли contention и что помогло